

UNIT - III

3

Context Free Grammar (CFG) and Context Free Language (CFL)

Syllabus

*Basic Elements of Grammar, Formal Definition of Context Free Grammar, Sentential form, Derivation and Derivation Tree/ Parse Tree, Context Free Language (CFL), Ambiguous Grammar, writing grammar for language. **Simplification of CFG** : Eliminating ϵ - productions, unit productions, useless production and useless symbols. **Normal Forms** : Chomsky Normal Form, Greibach Normal Form, Pumping Lemma for CFG, Closure properties of CFL, Decision properties of CFL, Chomsky Hierarchy, Cock-Younger-Kasami Algorithm.*

Contents

3.1	Basic Elements of Grammar	
3.2	Formal Definition of Context Free Grammar	
3.3	Sentential Form	Aug.-17, Marks 3
3.4	Context Free Language (CFL)	Dec.-05, 06, 07, 08, 09, 10, 13, 14, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, Marks 12
3.5	Derivation	May-15, Marks 4
3.6	Parse Tree	
3.7	Ambiguous Grammar	May-07, 11, 12, 14, Dec.-05, 12, 13, 14, Marks 8
3.8	Simplification of CFG	Dec.-16, 18, 19, Aug.-17, Oct.-18, 19, Marks 8
3.9	Normal Forms	Dec.-05, 10, 13, 14, 15, 16, 17, 19, May-08, 10, 13, 14, 18, Oct.-18, Marks 12
3.10	Pumping Lemma for CFG	
3.11	Closure Properties of CFL	May 09, Dec.-18, Marks 6
3.12	Decision Properties of CFL	Dec.-11, Marks 5
3.13	Chomsky Hierarchy	May-06, Dec.-11, Marks 8
3.14	Cock-Younger-Kasami Algorithm	
3.15	Applications of CFG	May-06, 11, Dec.-06, 13, Oct.-16, 19, Marks 6
3.16	Case Studies	
3.17	Fill in the Blanks	
3.18	Multiple Choice Questions	

3.1 Basic Elements of Grammar

A grammar is a 4 tuple $G = (V, T, P, S)$ where V is a finite set of **non terminal symbols**, T is a finite set of **terminal symbols**, P is a set of **production rules** and S is a **start symbol**.

The P is of the form $A \rightarrow B$ where A and $B \in (V \cup T)^*$.

The grammar is normally represented using derivation tree.

For example

The production rules for defining natural language are as follows -

Sentence $\rightarrow$ Noun verb

Noun $\rightarrow$ Rama | Sita | Gopal

Verb $\rightarrow$ sings | eats | dances

A derivation tree can also be built using the production rules.

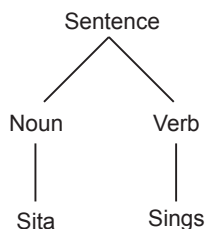


Fig. 3.1.1 Phrase tree for the string Sita Sings

3.2 Formal Definition of Context Free Grammar

The context free grammar can be formally defined as a set denoted by $G = (V, T, P, S)$ where V and T are set of non terminals and terminals respectively. P is set of production rules, where each production rule is in the form of

Non terminal $\rightarrow$ Non terminals

or Non terminal $\rightarrow$ Terminals

S is a start symbol.

For example

$P = \{ S \rightarrow S + S$

$S \rightarrow S * S$

$S \rightarrow (S)$

$S \rightarrow 4 \}$

If the language is $4 + 4 * 4$ then we can use the production rules given by P. The start symbol is S. The number of non terminals in the rules P is one and the only non terminal i.e. S. The terminals are +, *, (,) and 4.

We are using following conventions.

1. The capital letters are used to denote the non terminals.
2. The lower case letters are used to denote the terminals.

Example : The formation of production rules for checking syntax of any English statement is

SENTENCE $\rightarrow$ NOUN VERB

NOUN $\rightarrow$ Rama | Sita | Gopal

VERB $\rightarrow$ goes | writes | sings

Thus if want to derive a string "Rama sings" then we can follow the above rules.

3.3 Sentential Form

SPPU : Aug.-17, Marks 3

Consider $G = (V, T, P, S)$ be a context free grammar then, we can derive a string w from it. This w can obtained from $(VUT)^*$ where V denotes the set of non-terminal symbols and T denotes the set of terminal symbols. The derivation of w from start symbol S can be written as $S \xRightarrow{*} w$ which is called as **sentential form**. If $S \xRightarrow{*}_{lm} w$ then w is a **left-sentential form**.

If $S \xRightarrow{*}_{rm} w$ then w is a **right-sentential form**.

For example

Consider the grammar,

$S \rightarrow S + S \mid S * S \mid 4$

Then for deriving the string $w = 4 + 4 * 4$ we use above grammar as -

$S \Rightarrow S * S \Rightarrow S + S * S \Rightarrow 4 + S * S \Rightarrow 4 + 4 * S \Rightarrow 4 + 4 * 4$
 $lm \quad \quad lm \quad \quad lm \quad \quad lm \quad \quad lm$

Here $S + S * S$ is called left-sentential form.

$S \Rightarrow S + S \Rightarrow S + S * S \Rightarrow S + S * 4 \Rightarrow S + 4 * 4 \Rightarrow 4 + 4 * 4$
 $rm \quad \quad rm \quad \quad rm \quad \quad rm \quad \quad rm$

$S + S * 4$ is called right sentential form. Thus we can obtain the desired language L by certain rules. Let us now solve some examples for derivation of CFG.

Review Question

1. Write in brief about "Sentential form" with reference to context free grammar.

SPPU : Aug.-17, In Sem, Marks 3

3.4 Context Free Language (CFL)

SPPU : Dec.-05, 06, 07, 08, 09, 10, 13, 14, 17, May-08, 09, 10, 13, 14, 19, Aug.-17, Oct.-16, 18, 19, Marks 12

The language of grammar can be defined by creating the production rules for the given condition. The language generated by context free grammar is called as Context Free Language (CFL).

Let us understand the CFL with the help of various examples.

Example 3.4.1 Construct the CFG for the language having any number of a 's over the set $\Sigma = \{a\}$.

SPPU : Oct.-19, In Sem, Marks 3

Solution : As we know the regular expression for above mentioned language is

$$\text{r.e.} = a^*$$

Let us build the production rules for the same,

$$S \rightarrow aS \quad \text{rule 1}$$

$$S \rightarrow \epsilon \quad \text{rule 1}$$

Now if want "aaaaa" string to be derived we can start with start symbols.

S

aS

$aaS \quad \therefore \quad \text{rule 1}$

$aaaS \quad \text{rule 1}$

$aaaaS \quad \text{rule 1}$

$aaaaaS \quad \text{rule 1}$

$aaaaa\epsilon \quad \text{rule 2}$

$= aaaaa$

The $\text{r.e.} = a^*$ suggest a set of $\{\epsilon, a, aa, aaa, \dots\}$. We can have a null string simply because S is a start symbol, and rule 2 gives $S \rightarrow \epsilon$.

Example 3.4.2 Try to recognize the language L for given CFG.

$$G = [\{S\}, \{a, b\}, P, \{S\}]$$

$$\text{where } P = \left\{ \begin{array}{l} S \rightarrow aSb \\ S \rightarrow ab \end{array} \right\}$$

SPPU : Oct.-19, In Sem, Marks 4

Solution : Since $S \rightarrow a S b \mid a b$ is a rule. $\mid$ indicates the 'or' operator.

$$S \rightarrow a S b$$

If this rule can be recursively applied then,

$$\begin{array}{c} S \\ a S b \\ \downarrow \\ a a S b b \\ \downarrow \\ a a a S b b b \end{array}$$

and if finally we can put $S \rightarrow ab$ then it becomes $aaaa bbbb$. Thus we can have any number of a 's first then equal number of b 's following it. Hence we can guess the language as $\{L = a^n b^n \text{ where } n \geq 1\}$. The only way to recognize the language is to try out various strings from the given production rules. Simply by observing the derived strings, one can find out the language getting generated from given CFG.

Example 3.4.3 Construct the CFG for the regular expression $(0+1)^*$.

SPPU : Dec.-05, Marks 8

Solution : The CFG can be given by,

$$\begin{aligned} P &= \{S \rightarrow 0S \mid 1S \\ &\quad S \rightarrow \epsilon\} \end{aligned}$$

The rules are in combination of 0's and 1's with the start symbol. Since $(0+1)^*$ indicates $\{\epsilon, 0, 1, 01, 10, 00, 11, \dots\}$ in this set ϵ is a string. So in the rules we can set the rule $S \rightarrow \epsilon$.

Example 3.4.4 Construct a grammar for the language containing strings of at least two a 's.

Solution : Let $G = (V, T, P, S)$

where $V = \{S, A\}$

$T = \{a, b\}$

$P = \{S \rightarrow Aa Aa A$

$A \rightarrow aA \mid bA \mid \epsilon\}$

The rule $S \rightarrow AaAaA$ is something in which the two a 's are maintained since at least two a 's should be there in the strings. And $A \rightarrow aA \mid bA \mid \epsilon$ gives any combination of a 's and b 's i.e. this rule gives the strings of $(a + b)^*$.

Thus the logic for this example will be

(any thing) a (any thing) a (any thing)

So before a or after a there could be any combination of a 's and b 's.

Example 3.4.5 Construct a grammar generating

$$L = w c w^T \text{ where } w \in \{a, b\}^*$$

Solution : The strings which can be generated for given L is
 $\{aacaa, bcb, abcba, bacab \dots\}$

The grammar could be

$$S \rightarrow a S a$$

$$S \rightarrow b S b$$

$$S \rightarrow c$$

Since the language $L = w c w^T$ where $w \in (a + b)^*$

Hence $S \rightarrow a S a$ or $S \rightarrow b S b$. The string $abcba$ can be generated from given production rules as

S
 $a S a$
 $a b S b$
 $a b c b a$

a	b	c	b	a
---	---	---	---	---

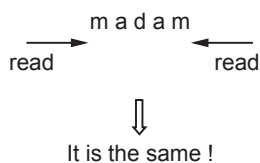
a	b	c	b	a
---	---	---	---	---

a	b	c	b	a
---	---	---	---	---

Thus any of this kind of string could be derived from the given production rules.

Example 3.4.6 Construct CFG for the language L which has all the strings which are all palindrome over $\Sigma = \{a, b\}$.

Solution : As we know the strings are palindrome if they possess same alphabets from forward as well as from backward.



For example, the string "madam" is a palindrome because

Since the language L is over $\Sigma = \{a, b\}$. We want the production rules to be build a's and b's. As ϵ can be the palindrome, a can be palindrome even b can be palindrome. So we can write the production rules as

$$G = (\{S\}, \{a, b\}, P, S)$$

P can be $S \rightarrow a S a$

$$S \rightarrow b S b$$

$$S \rightarrow a$$

$$S \rightarrow b$$

$$S \rightarrow \epsilon$$

The string abaaba can be derived as

S	
a S a	⊥ b a a b ⊥
a b S b a	⊥ b a a b ⊥
a b a S a b	a ⊥ b ⊥ ⊥ b a
a b a ε a b	a ⊥ b ⊥ ⊥ b ⊥
a b a a b a	⊥ b ⊥ ⊥ b ⊥

which is a palindrome.

Example 3.4.7 Construct CFG which consists of all the strings having at least one occurrence of 000.

Solution : The CFG for this language can be equivalent to r.e. (any thing) (000) (anything)

Thus r.e. = $(0+1)^*000(0+1)^*$

Let us build the production rules as

$S \rightarrow ATA$

$A \rightarrow 0A \mid 1A \mid \epsilon$

$T \rightarrow 000$

Example 3.4.8 Construct CFG for the language in which there are no consecutive b's, the strings may or may not have consecutive a's.

Solution : $S \rightarrow aS \mid bA \mid a \mid b \mid \epsilon$

$A \rightarrow aSa \mid \epsilon$

In the above rules, there is no condition on occurrence of a's. But no consecutive b's are allowed. Note that in the rule $S \rightarrow bA$, and A gives all the strings which are starting with letter a.

Thus $G = (\{S, A\}, \{a, b\}, P, S)$

Let us derive the string

Derivation	Input	Productions
S		
a S	⊥ a b a b	$S \rightarrow a S$
a a S	⊥ ⊥ b a b	$S \rightarrow a S$
a a b A	⊥ ⊥ b a b	$S \rightarrow b A$
a a b a S	⊥ a b ⊥ b	$A \rightarrow a S$
a a b a b	⊥ ⊥ b ⊥ b	$S \rightarrow b$

Example 3.4.9 Recognize the context free language for the given CFG.

$$S \rightarrow a B | b A$$

$$A \rightarrow a | a S | b A A$$

$$B \rightarrow b | b S | a B B$$

SPPU : Aug.-17, In Sem, Marks 3

Solution : To find the language denoted by given CFG we try to derive the rules and get various strings. Then after observing those strings we come to know, which language it is denoting. Let us rewrite all those rules and number them

$$S \rightarrow a B \quad \text{rule 1}$$

$$S \rightarrow b A \quad \text{rule 2}$$

$$A \rightarrow a \quad \text{rule 3}$$

$$A \rightarrow a S \quad \text{rule 4}$$

$$A \rightarrow b A A \quad \text{rule 5}$$

$$B \rightarrow b \quad \text{rule 6}$$

$$B \rightarrow b S \quad \text{rule 7}$$

$$B \rightarrow a B B \quad \text{rule 8}$$

Now let us apply the rules randomly starting with start symbol.

S

$$a B \quad \text{rule 1}$$

$$a a B B \quad \text{rule 8}$$

$$a a b S B \quad \text{rule 7}$$

$$a a b b A B \quad \text{rule 2}$$

$$a a b b a B \quad \text{rule 3}$$

$$a a b b a b S \quad \text{rule 7}$$

$$a a b b a b b A \quad \text{rule 2}$$

$$a a b b a b b a \quad \text{rule 3}$$

We have got some string as "aabbabba". Let us try out something else.

S

$$b A \quad \text{rule 2}$$

$$b b A A \quad \text{rule 5}$$

$$b b a S A \quad \text{rule 4}$$

$$b b a a B A \quad \text{rule 1}$$

$$b b a a b A \quad \text{rule 6}$$

$$b b a a b a \quad \text{rule 3}$$

Now we have got "bbaaba" such a string !

Thus by obtaining more and more strings by applying more and more rules randomly will help us to find out what language it indicates. Observe the strings generated carefully, we can draw a conclusion as these strings contain equal number of a 's and equal number of b 's. Even you can try out various rules to obtain some more strings. And see that it is a language L containing all the strings having equal number of a 's and b 's.

Example 3.4.10 Construct CFG for the language containing at least one occurrence of double a .

SPPU : Dec.-17, End Sem, Marks 4

Solution : The CFG can be built with a logic as : one rule we will built for double a i.e. $A \rightarrow aa$.

And the other rule we will built for any number of a 's and b 's in any combination.

i.e. $B \rightarrow aB|bB|\epsilon$ which is always equivalent to $(a + b)^*$ {you can note it as golden rule !}

If we combine both of these rules we can get the desired context free grammar.

$$S \rightarrow BAB \Rightarrow (\text{anything}) \left(\begin{array}{c} \text{one occurrence} \\ \text{of double } a \end{array} \right) (\text{anything})$$

$$A \rightarrow aa$$

$$B \rightarrow aB|bB|\epsilon$$

Thus the CFG contains $V = \{A, B, S\}$ $T = \{a, b\}$. The start symbol is S .

Let us derive "abaab"

S

BAB

$aBAB \quad abaab \quad B \rightarrow aB$

$abBAB \quad abbaab \quad B \rightarrow bB$

$abAB \quad abbaab \quad B \rightarrow \epsilon$

$abaaB \quad abbaab \quad A \rightarrow aa$

$abaabB \quad abbaab \quad B \rightarrow bB$

$abaabe \quad abbaab \quad B \rightarrow \epsilon$

$= abaab \quad abbaab$

Example 3.4.11 Construct CFG for the language containing all the strings of different first and last symbols over $\Sigma = \{0, 1\}$.

Solution : Since the problem statement says as if the string starts with 0 it should end with 1 or if the string starts with 1 it should end with 0.

$$S \rightarrow 0 A 1 \mid 1 A 0$$

$$A \rightarrow 0 A \mid 1 A \mid \varepsilon$$

Thus clearly, in the above CFG different start and end symbols are maintained. The non terminal $A \rightarrow 0A \mid 1A \mid \varepsilon$ indicates $(0 + 1)^*$. Thus the given CFG is equivalent to the regular expression $[0(0+1)^*1 + 1(0+1)^*0]$.

Example 3.4.12 Construct the CFG for the language $L = a^n b^{2n}$ where $n \geq 1$.

Solution : As in some previous example we have seen the case of $a^n b^n$. Now the number of b's are doubled. So we can write

$$S \rightarrow a S b b \mid a b b$$

It is as simple as this !

Example 3.4.13 Construct the production rules for defining a language $L = \{a^x b^y \mid x \neq y\}$.

SPPU : Dec.-06, Marks 8

Solution : This is the language in which all the a's must appear before all the b's. But total number of a's must not be equal to total number of b's. Either number of a's must be equal to number of b's or number of b's must be equal to number of a's. The rule

$$S \rightarrow aSb$$

gives us equal number of a's must be followed by equal number of b's. But according to problem statement we need more number of a's either or more number of b's. Hence the required production rules can be -

$$S \rightarrow aSb \mid R1 \mid R2$$

$$R1 \rightarrow aR1 \mid a$$

$$R2 \rightarrow bR2 \mid b$$

where S is a start symbol.

Example 3.4.14 Find the CFG for the regular expression

$$(110 + 11)^*(10)^*$$

Solution : Let

$$\begin{array}{lcl} \text{regular expression} & = & (110 + 11)^* (10)^* \\ \text{Can be represented} & & \downarrow \quad \downarrow \\ \text{by the non} & & A \quad B \\ \text{terminal} & & \end{array}$$

If we assume S as start symbol then,

$$S \rightarrow AB$$

Now for each non terminal A and B we can define production rules as

$$A \rightarrow 110A \mid 11A \mid \epsilon$$

$$B \rightarrow 10B \mid \epsilon$$

Hence, finally the CFG for given r.e. can be

$$S \rightarrow AB$$

$$A \rightarrow 110A \mid 11A \mid \epsilon$$

$$B \rightarrow 10B \mid \epsilon$$

Example 3.4.15 Build a CFG for generating the integers.

Solution : The integer is any numerical value without decimal place which can be either signed or unsigned. Hence the CFG can be -

$$S \rightarrow GI$$

$$G \rightarrow + \mid -$$

$$I \rightarrow DI \mid D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \dots \mid 9$$

Here G is for denoting sign, I denotes integer and D denotes digits from 0 to 9. To understand the rules let's derive these rules for some integer.

Let, - 21 can be derived as

$$S \rightarrow GI$$

$$- \quad I \quad \quad G \rightarrow -$$

$$- \quad DI \quad \quad I \rightarrow DI$$

$$- \quad 2I \quad \quad D \rightarrow 2$$

$$- \quad 2D \quad \quad I \rightarrow D$$

$$- \quad 21 \quad \quad D \rightarrow 1$$

Example 3.4.16 Build a CFG for the language

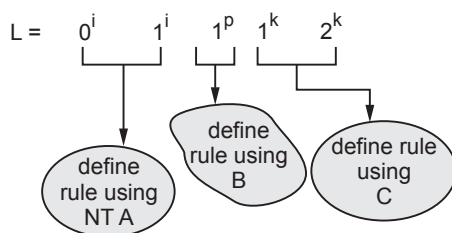
$$L = \{0^i 1^j 2^k \mid j > i + k\}.$$

Solution : As the language $L = 0^i 1^j 2^k$ such that $j > i + k$.

Hence we can rewrite L as

$$L = 0^i \underbrace{1^i 1^k 1^p}_{\substack{\text{This will be} \\ \text{greater than} \\ i+k}} 2^k$$

We can again rewrite L for simplification as -



$$A \rightarrow 0 A 1 \mid \epsilon$$

$$B \rightarrow 1 B \mid 1$$

$$C \rightarrow 1 C 2 \mid \epsilon$$

Hence the CFG can be

$$S \rightarrow A B C$$

$$A \rightarrow 0 A 1 \mid \epsilon$$

$$B \rightarrow 1 B \mid 1$$

$$C \rightarrow 1 C 2 \mid \epsilon$$

Example 3.4.17 Build a CFG for the language $L = \{0^i 1^j 2^k \mid i=j\}$

Solution : As $i = j$ and there is no condition on k . We can say that k is an arbitrary value. Then we rewrite L as

$$\therefore S \rightarrow A B$$

$$A \rightarrow 0 A 1 \mid \epsilon$$

$$B \rightarrow 2 B \mid \epsilon$$

Example 3.4.18 Obtain CFG for the language $L = \{0^i 1^j 2^k \mid j \leq k\}$.

Solution : Here $j \leq k$. That means the language L has two variations.

$$L_1 = 0^i 1^j 2^j$$

$$L_2 = 0^i 1^j 2^k \text{ where } k > j.$$

$$\text{Hence } L_2 = 0^i 1^j 2^j \cdot 2^k$$

Hence the required CFG can be

$$S \rightarrow A B$$

$$A \rightarrow 0 A \mid \epsilon$$

$$B \rightarrow 1 B 2 C \mid 12$$

$$C \rightarrow 2 C \mid \epsilon$$

Example 3.4.19 Give CFG for set of odd length settings in $\{0, 1\}^*$ with middle symbol '1'.

SPPU : May-14, Marks 4

Solution : $S \rightarrow 0S0 \mid 0S1 \mid 1S0 \mid 1S1 \mid 1$

Consider the string 01110 for derivation

S

0S0

01S10

01110

Example 3.4.20 If the grammar G is given by production

$S \rightarrow aSa \mid bSb \mid aa \mid bb \mid \epsilon$ show that

- Any string in $L(G)$ is of length $2n$, $n \geq 0$
- The number of strings of length $2n$ is 2^n

SPPU : Dec.-13, Marks 2

Solution : The CFG denotes the language L of even length in which the two middle symbols equal. Hence it will generate the strings such as

$L(G) = \{ \epsilon, aa, bb, aaaa, aaab, baab, baaa, abba, abbb, bbbb, bbba, \dots \}$

- If $n = 0$, the string is ϵ .
- If $n = 1$, the string of length 2 (i.e. 2×1) are aa and bb (i.e. $2^n = 2$).

Example 3.4.21 Describe the language generated by each of these grammars. Justify your answer with an example.

1) $S \rightarrow aSa \mid bSb \mid \epsilon$

2) $S \rightarrow aSa \mid bSb \mid a \mid b$

3) $S \rightarrow aSb \mid bSa \mid \epsilon$

4) $S \rightarrow SS \mid bS \mid a$

SPPU : Dec.-10, Marks 12

Solution :

1) L = The language containing all strings of even length palindrome.

S	$S \rightarrow aSa$
aSa	$S \rightarrow bSb$
abSba	$S \rightarrow \epsilon$
abba	

- 2) L = The language containing all strings of odd length palindrome.

S	$S \rightarrow aSab$
aSb	$S \rightarrow bSb$
abSba	$S \rightarrow a$
ababa	

- 3) L = The language containing all strings which contains equal number of a's and b's. But having different first and last symbol.

S	$S \rightarrow aSb$
aSb	$S \rightarrow bSa$
abSab	$S \rightarrow \epsilon$
abab	

- 4) L = The language containing all strings that end with letter a.

S	$S \rightarrow SS$
SS	$S \rightarrow bS$
bSS	$S \rightarrow a$
baS	$S \rightarrow a$
baa	

Example 3.4.22 Write a CFG for generating identifiers in higher-level languages such as 'C'.
Identifiers can be defined by the regular expression (letter). (letter | digit)*.

SPPU : May-13, Marks 4

Solution :

$$S \rightarrow LA$$

$$A \rightarrow LA \mid DA \mid \epsilon$$

$$L \rightarrow alblc \dots z$$

$$D \rightarrow 0|1|2 \dots 9$$

Example 3.4.23 In each case find context free grammar generating given language

a) The set of odd length strings in $\{a, b\}^*$ with middle symbol a.

b) The set of even length strings $\{a, b\}^*$ with the two middle symbols equal.

SPPU : Dec.-07, May-08, Marks 8

Solution :

a) Required CFG is -

$$S \rightarrow aSa$$

$$S \rightarrow aSb$$

$$S \rightarrow bSb$$

$$S \rightarrow bSa$$

$$S \rightarrow a$$

b) This language can be denoted by regular expression.

regular expression = $(a + b)^* (aa + bb) (a + b)^*$

Hence CFG can be given as

$$S \rightarrow aSa|aSb|bSb|bSa$$

$$S \rightarrow aa$$

$$S \rightarrow bb$$

Example 3.4.24 Describe the language generated by each of these grammars

a) The regular grammar with production :

$$S \rightarrow aA|bC|b$$

$$A \rightarrow aS|bB$$

$$B \rightarrow aC|bA|a$$

$$C \rightarrow aB|bS$$

b) The regular grammar with production :

$$S \rightarrow bS|aA|\epsilon$$

$$A \rightarrow aA|bB|b$$

$$B \rightarrow bS$$

SPPU : Dec.-07, 09, May-08, 09, Marks 8

Solution :

a) For obtaining the language generated by given CFG we will first draw FA for it.

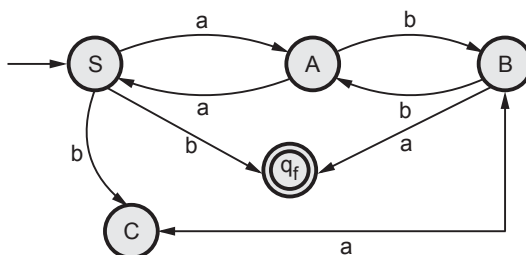


Fig. 3.4.1

We can merge q_f and state C. Hence the FA will be.

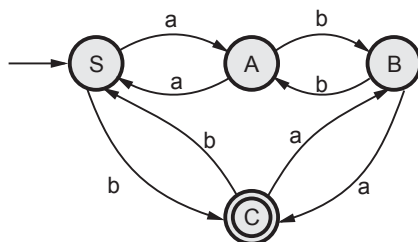


Fig. 3.4.2

We can derive some strings from above drawn FA. Those are {aab, bbb, b, aba, ...}

That means given language L accepts the strings containing even number of a's and odd number of b's.

- b) For obtaining the language generated by given CFG we will first draw FA for it.

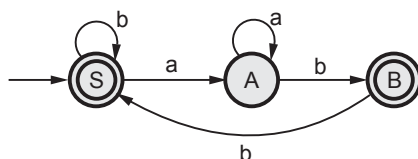


Fig. 3.4.3

The r.e. = $(a^*aa^*b(\epsilon+b))^* + b^*$

This regular expression denotes the strings made up of either any number of b's or the strings that contain the substring 'ab' but not contain the substrings 'aba' at all.

Example 3.4.25 Give the context free grammars for the following languages

$$a) L = \{0^m 1^n 0^{m+n} \mid m, n \geq 0\} \quad b) L = \{0^a 1^b 2^c \mid |a - c| = b\}$$

SPPU : Dec.-08, Marks 12

Solution :

a) Let, $L = \{0^m 1^n 0^{m+n}\}$

We will rewrite L as

$$= 0^m 1^n 0^n 0^m \quad \because n + m = m + n$$

Hence $P = \{$

$$S \rightarrow 0S0 \mid 0100$$

$$S \rightarrow 01A00$$

$$A \rightarrow 1A0 \mid \epsilon$$

$\}$

Consider the derivation for string 001000 as

S

0S0 $S \rightarrow 0S0$

001000 $S \rightarrow 0100$

b) Let, $L = \{0^a 1^b 2^c \mid a - c = b\}$

If we arrange $a = |b+c|$ then we can rewrite L as

$$= 0^{b+c} 1^b 2^c$$

$$= 0^c 0^b 1^b 2^c$$

Hence CFG can be

$$S \rightarrow 0S2 \mid 0012$$

$$S \rightarrow 00A12$$

$$A \rightarrow 0A1 \mid \epsilon$$

Let us derive the string 000000112222 as follows -

S $S \rightarrow 0S2$

0S2 $S \rightarrow 0S2$

00S22 $S \rightarrow 0S2$

000S222 $S \rightarrow 00A12$

00000A12222 $A \rightarrow 0A1$

000000A112222 $A \rightarrow \epsilon$

000000112222

$$\text{i.e. } 0^6 1^2 2^4 \in \{L = 0^a 1^b 2^c \mid |a - c| = b\}$$

Example 3.4.26 Give the context free grammar for the following languages

$$a) L = \{a^n b^{2n} \mid n > 1\} \quad b) L = \{a^m b^n \mid n > m\}$$

SPPU : Dec.-08, Marks 12

Solution :

$$a) \text{ Let, } L = \{a^n b^{2n} \mid n > 1\}$$

The production rules for the CFG of above language will be -

$$S \rightarrow aAbb$$

$$A \rightarrow aAbb \mid abb$$

$$b) \text{ Let, } L = \{a^m b^n \mid n > m\}$$

The L can be written as $L = a^m b^m b^K$ where $K \geq 1$.

Hence required CFG will be

$$S \rightarrow aSbB \mid \epsilon$$

$$B \rightarrow bB \mid b$$

Example 3.4.27 Give the context free grammars for the following languages.

a) $(011 + 1)^* (01)^*$

b) $0^i 1^{i+k} 0^k$ where $i, k \geq 0$.

SPPU : May-09, Marks 12, May-14, Marks 8

Solution : a) The CFG can be denoted by,

$$G = (\{S, A, B, C, D\}, \{0, 1\}, P, S)$$

where $P = \{$

$$S \rightarrow AB$$

$$A \rightarrow CA \mid \epsilon$$

$$C \rightarrow 011 \mid 1$$

$$B \rightarrow DB \mid \epsilon$$

$$D \rightarrow 01$$

$\}$

Let us derive a valid string $01110101 \in (011 + 1)^* (01)^*$

S

AB

CAB

CCAB

CCADB

CCADDB

011CADDB

0111ADDB

0111DDB

011101DB

01110101B

0111101

b) Let us rewrite the language L as

$$L = 0^i 1^{i+k} 0^k$$

$$L = 0^i 1^i 1^k 0^k$$

Hence required CFG $G = (\{S, A, B\}, \{0, 1\}, P, S)$

Hence

$$\begin{aligned} P &= \{S \rightarrow AB \\ &= A \rightarrow 0A1 \mid \epsilon \\ &= B \rightarrow 1B0 \mid \epsilon \\ &\} \end{aligned}$$

Example 3.4.28 Describe the language generated by each of these grammar and justify your answer with the example string derive from the grammar of the productions given below.

$$1) S \rightarrow aSa \mid bSb \mid aAb \mid bAa$$

$$A \rightarrow aAa \mid bAb \mid a \mid b \mid \epsilon$$

$$2) S \rightarrow bT \mid aT \mid \epsilon$$

$$T \rightarrow aS \mid bS$$

SPPU : May-10, Marks 12

Solution :

1) This is a language which is a combination of two language L_1 and L_2 , where L_1 denotes all the strings that are palindrome.

L_2 denotes all the strings containing the substrings as palindrome but their first and last letter are different.

$$\text{i.e. } L_2 = [a (\text{Palindrome string}) b + b (\text{palindrome string}) a]$$

2) To derive the string for this CFG we will draw a FA

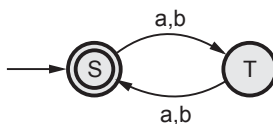


Fig. 3.4.4

Clearly, this FA suggest a language containing all the strings having even length.

Example 3.4.29 Find the context free grammar for the following languages.

$$i) L = \{a^n b^m \mid n \neq m\}$$

$$ii) \{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$$

SPPU : Dec.-06, Marks 8

Solution : i) $L = \{a^n b^m \mid n \neq m\}$: Refer example 3.4.26 (b).

ii) $\{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$:

Here L is a combination of L_1 and L_2 where

$$L_1 = \{a^n b^m c^k \mid n = m\}$$

$$L_2 = \{a^n b^m c^k \mid m \leq k\}$$

The CFG for L_1 can be

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow cB \mid \epsilon$$

The CFG for L_2 can be

$$S \rightarrow PQ$$

$$P \rightarrow aP \mid \epsilon$$

$$Q \rightarrow bQcR \mid bc$$

$$R \rightarrow cR \mid \epsilon$$

Hence the CFG for L will be

$$S \rightarrow XY$$

$$X \rightarrow AB$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow cB \mid \epsilon$$

$$Y \rightarrow PQ$$

$$P \rightarrow aP \mid \epsilon$$

$$Q \rightarrow bQcR \mid bc$$

$$R \rightarrow cR \mid \epsilon$$

Example 3.4.30 Give context free grammars for the following languages.

i) $L = \{S \rightarrow aAb, A \rightarrow aAbA \mid \epsilon, x \mid x \in \{a, b\}^* \text{ with strings of starting with 'a' and ending 'b'}\}$

ii) $L = \{x \mid x \in \{a, b\}^* \text{ with strings of even length palindrome}\}$.

SPPU : Dec.-14, In Sem, Marks 4

Solution : i) $S \rightarrow aAb, A \rightarrow aAbA \mid \epsilon$.

ii)

Step 1 : As we know the strings are palindrome if they possess same alphabets from forward as well as from backward.

Step 2 : Let $G = (\{S\}, \{a, b\}, P, S)$

Step 3 : The string abaaba can be derived as

P can be $S \rightarrow aSa$ S
 $S \rightarrow bSb$ aSa $\#baab\#$

$$\begin{array}{lll}
 S \rightarrow a & a b S b a & \emptyset b a a b \emptyset \\
 S \rightarrow b & a b a S a b & a \emptyset b \emptyset \emptyset b a \\
 S \rightarrow \varepsilon & a b a \varepsilon a b & a \emptyset b \emptyset \emptyset b \emptyset \\
 & a b a a b a & \emptyset b \emptyset \emptyset b \emptyset
 \end{array}$$

Example 3.4.31 Give context free grammars for the following languages :

i) $L = \{x \mid x \in (,)^*\}$ with strings having well-formed parentheses (WFP)

ii) $L = \{a^m b^n c^{m+n} \mid m, n \geq 0\}$

SPPU : Oct.-16, In sem, Marks 6

Solution :

i) The production rule are

$$\begin{aligned}
 P &= \{ \\
 &\quad S \rightarrow () S \mid () S \mid (S) \mid () \\
 &\quad \}
 \end{aligned}$$

ii) We will first rewrite the given L as

$$L = a^m b^n c^n c^m$$

$$\begin{aligned}
 \text{Hence } P &= \{ \\
 &\quad S \rightarrow aSc \mid abcc \\
 &\quad S \rightarrow abAcc \\
 &\quad A \rightarrow bAc \mid \varepsilon \\
 &\quad \}
 \end{aligned}$$

Consider the derivation for string

aabccc

S

aSc

aabccc

Example 3.4.32 Write context free grammar for the following language $0(0+1)^*01(0+1)^*1$

SPPU : Aug.-17, In sem, Marks 4

Solution :

$$\begin{aligned}
 P &\rightarrow \{ \\
 &\quad S \rightarrow 0A01A1 \\
 &\quad A \rightarrow 0A \mid 1A \mid \varepsilon \\
 &\quad \}
 \end{aligned}$$

Example 3.4.33 Write the CFG for following language.

$$L = \{a^{m+n}b^m c^n \mid n, m \geq 0\}$$

SPPU : Oct.-18, In Sem, Marks 4

Solution : We will rewrite language L as

$$L = a^n a^m b^m c^n$$

Hence $P = \{$

$$S \rightarrow aSc \mid abcc$$

$$S \rightarrow abAcc$$

$$A \rightarrow bAc \mid \epsilon$$

$\}$

Example 3.4.34 Write the grammar generating all strings consisting of a's and b's with at least two a's.

SPPU : Oct.-18, In Sem, Marks 2

Solution : We will write regular expression for this language

$$\text{r.e.} = (a + b)^* a (a + b)^* a (a + b)^*$$

The production rules $(a + b)^*$

$$A \rightarrow aA \mid bA \mid \epsilon$$

Hence production rules for regular expression

$$S \rightarrow A \times A \times A$$

$$X \rightarrow a$$

$$A \rightarrow aA \mid bA \mid \epsilon$$

Example 3.4.35 Write CFG for following language.

$$L = \{a^n b^m a^n \mid n \geq 0, m \geq 1\}$$

SPPU : Oct.-18, In Sem, Marks 4

Solution : The CFG can be given by following production rule.

$$P = \{$$

$$S \rightarrow aSa \mid bB$$

$$B \rightarrow bB \mid \epsilon$$

$\}$

Simulation :

The string aabbbaa can be derived as

S

aSa

aaSaa

aabBaa

aabbBaa

aabbbbBaa

aabbbaa

Example 3.4.36 Write CFG for the following languages.i) $L = \{0^i 1^j 0^k \mid j > i + k\}$ (Refer example 3.4.16)ii) $L = \{0^i 1^j 2^k \mid i = j + k\}$ **SPPU : May-19, End Sem, Marks 4****Solution : ii)** Let,

$$L = \{0^i 1^j 2^k \mid i = j + k\}$$

We can rewrite L as

$$\begin{aligned} L &= 0^j 0^k 1^j 2^k \\ &= 0^k 0^j 1^j 2^k \end{aligned}$$

The production rules are

$$\begin{aligned} S &\rightarrow 0X_2 \mid 02 \\ X &\rightarrow 0X_1 \mid \epsilon \end{aligned}$$

3.5 Derivation

SPPU : May-15, Marks 4

- The production rules are used to derive certain strings. If $G = (V, T, P, S)$ be some context free grammar then generation of some language using specific rules is called **derivation**.
- As we know, derivation for any string means replacement of non terminal by its appropriate definition. There may be a situation, in which there are many non terminals. Then which non terminal should be replaced by its definition is sometimes confusing to decide.
- Hence we normally apply **two methods of deriving**.
- The **leftmost derivation** is a derivation in which the leftmost non terminal is replaced first from the sentential form.
- The **rightmost derivation** is a derivation in which rightmost non terminal is replaced first from the sentential form.

Let us see how it works.

For example

$$S \rightarrow XYX$$

$$S \rightarrow aYX$$

$$S \rightarrow abX$$

$$S \rightarrow aba$$

Leftmost derivation

$$S \rightarrow XYX$$

$$S \rightarrow XYa$$

$$S \rightarrow Xba$$

$$S \rightarrow aba$$

Rightmost derivation

Note that we have replaced first X from left to right in leftmost derivation and XYX the last X i.e. the rightmost symbol.

Actually, we may use leftmost derivation or rightmost derivation we get the same string. The type of derivation does not affect on getting of a string.

Example 3.5.1 Derive the string "aabbabba" for leftmost derivation and rightmost derivation using a CFG given by,

$$S \rightarrow aB|bA$$

$$A \rightarrow a|aS|bAA$$

$$B \rightarrow b|bS|aBB$$

Solution : Let us see the leftmost derivation first,

S	
aB	$S \rightarrow aB$
aaBB	$B \rightarrow aBB$
aabB	$B \rightarrow b$
aabbS	$B \rightarrow bS$
aabbabB	$S \rightarrow aB$
aabbabS	$B \rightarrow bS$
aabbabbA	$S \rightarrow bA$
aabbabba	$A \rightarrow a$

is derived. Now let us solve using rightmost derivation.

S	
aB	$S \rightarrow aB$
aaBB	$B \rightarrow aBB$
aaBbS	$B \rightarrow bS$
aaBbbA	$S \rightarrow bA$
aaBbba	$A \rightarrow a$
aabSbba	$B \rightarrow bS$
aabbAbba	$S \rightarrow bA$
aabbabba	$A \rightarrow a$

is a derivation.

Example 3.5.2 Derive the string 1000111 for leftmost and rightmost derivation using CFG.

$G = (V, T, P, S)$ where

$V = \{S, T\}$

$T = \{0, 1\}$

$P = \{ S \rightarrow T00T$

$T \rightarrow 0T \mid 1T \mid \epsilon \}$

Solution : The leftmost derivation can be

S

T00T $S \rightarrow T00T$

1T00T $T \rightarrow 1T$

10T00T $T \rightarrow 0T$

10 ϵ 00T $T \rightarrow \epsilon$

1000T

10001T $T \rightarrow 1T$

100011T $T \rightarrow 1T$

1000111T $T \rightarrow 1T$

1000111 ϵ $T \rightarrow \epsilon$

1000111

Rightmost derivation -

S

T00T

T001T $T \rightarrow 1T$

T0011T $T \rightarrow 1T$

T00111T $T \rightarrow 1T$

T00111 ϵ $T \rightarrow \epsilon$

T00111

1T00111 $T \rightarrow 1T$

10T00111 $T \rightarrow 0T$

10 ϵ 00111 $T \rightarrow \epsilon$

1000111

Example 3.5.3 Write the CFG for language $L = \{0^i 1^j 0^k \mid j > i + k\}$. Show the derivation of the string '0111100'.

SPPU : May-15, End Sem, Marks 4

Solution : The CFG can be

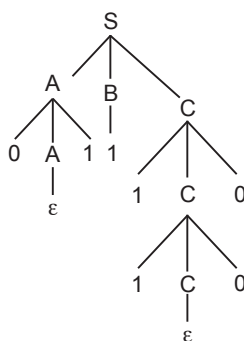
$$S \rightarrow ABC$$

$$A \rightarrow 0A1 \mid \epsilon$$

$$B \rightarrow 1B \mid 1$$

$$C \rightarrow 1C0 \mid \epsilon$$

Derivation of 0111100



3.6 Parse Tree

Parse trees is a graphical representation for the derivation of the given production rules for a given CFG. It is the simple way to show how the derivation can be done to obtain some string from given set of production rules. The parse tree is also called derivation tree.

Properties of Parse Tree

Following are properties of any parse tree -

1. The root node is always a node indicating start symbol.
2. The derivation is read from left to right.
3. The leaf nodes are always terminal nodes.
4. The interior nodes are always the non terminal nodes.

For example

$S \rightarrow bSb|a|b$ is a production rule. The S is a start symbol.

The above tree is a derivation tree drawn for deriving a string bbabb. By simply reading the leaf nodes we can obtain the desired string. The same tree can also be denoted by,

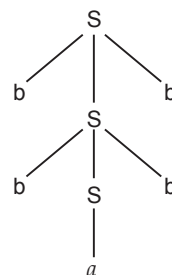
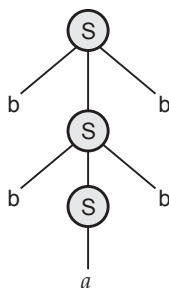


Fig. 3.6.1 Parse tree



Problems on parse tree

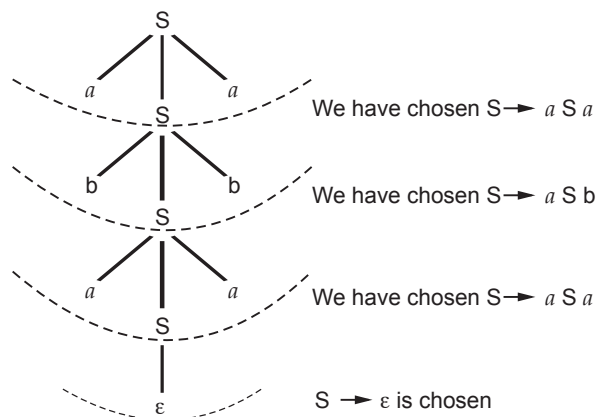
Example 3.6.1 Draw a parse tree for the string *abaaba* for the CFG given by,

G where $P = \{S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow a|b|\epsilon\}$

Solution :



Example 3.6.2 Construct the parse tree for the string *aabbabba* from the CFG given by

$S \rightarrow aB|bA$

$A \rightarrow a|aS|bAA$

$B \rightarrow b|bS|aBB$

Solution : To draw a tree we will first try to obtain derivation for the string *aabbabba*

S

aB

$S \rightarrow aB$

a

aBB

$S \rightarrow aBB$

aa

bS

B

$B \rightarrow bS$

aab

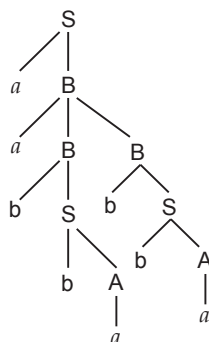
bA

B

$S \rightarrow bA$

$aabb\ a\ \boxed{B}$	$A \rightarrow a$
$aabba\ \boxed{bS}$	$B \rightarrow bS$
$aabbab\ \boxed{bA}$	$S \rightarrow bA$
$aaBbabb\ \boxed{a}$	$A \rightarrow a$

Now let us draw a tree.



3.7 Ambiguous Grammar

SPPU : May-07, 11, 12, 14, Dec.-05, 12, 13, 14, Marks 8

The grammar can be derived in either leftmost derivation or rightmost derivation. One can draw a derivation tree called as **parse tree** or **syntax tree** based on these derivations. The parse tree has to be unique even though the derivation is leftmost or rightmost.

• Definition

If there exists more than one parse trees for a given grammar, that means there could be more than one leftmost or rightmost derivation possible and then that grammar is said to be ambiguous grammar.

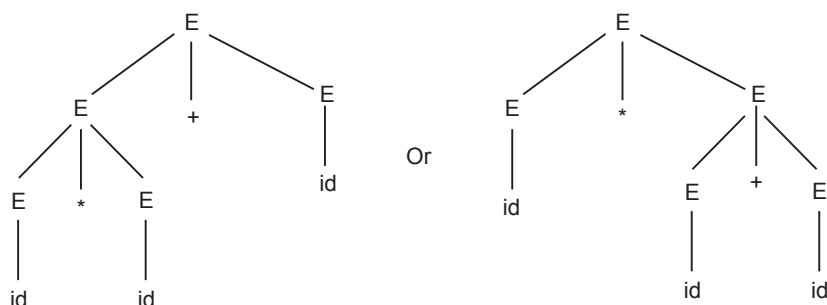
• Example

For example : The CFG given by $G = (V, T, P, S)$

where

$$\begin{aligned}
 V &= \{E\} \\
 T &= \{id\} \\
 P &= \{E \rightarrow E + E \\
 &\quad E \rightarrow E * E \\
 &\quad E \rightarrow id \} \\
 S &= \{E\}
 \end{aligned}$$

Now if the string is $\text{id} * \text{id} + \text{id}$ then we can draw the two different parse trees indicating our $\text{id} * \text{id} + \text{id}$.



Thus the above grammar is an ambiguous grammar. Let us solve some exercise on it.

Problems on ambiguity of grammar

Example 3.7.1 Check whether the given grammar is ambiguous or not.

$S \rightarrow iC t S$

$S \rightarrow iC t S e S$

$S \rightarrow a$

$C \rightarrow b$

Solution : To check whether given grammar is ambiguous or not we will have some string for derivation tree such as ibtibtibtaea .

Let us draw the derivation tree, for the string ibtibtibtaea

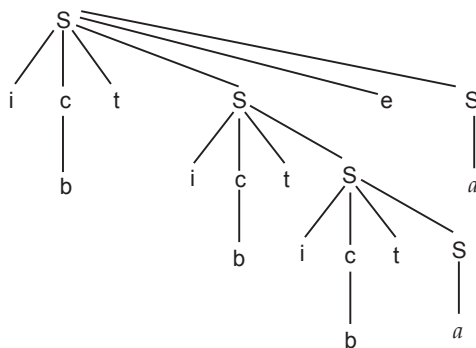


Fig. 3.7.1 (a) For example 3.7.1

or

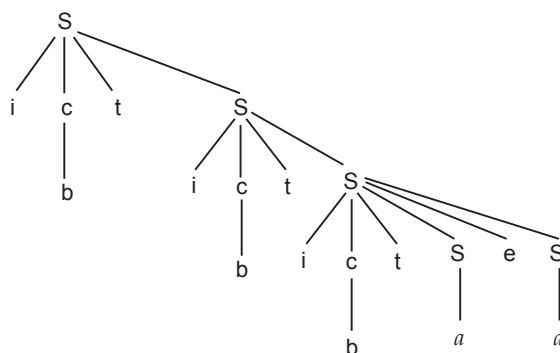


Fig. 3.7.1 (b) For example 3.7.1

Thus we have got more than two parse trees. Hence the given grammar is ambiguous.

Example 3.7.2 Consider the grammar $G = (V, \Sigma, R, S)$.

where $V = \{S, A\}$

$\Sigma = \{a, b\}$

$R = \{ S \rightarrow AA$

$A \rightarrow AAA$

$A \rightarrow a$

$A \rightarrow bA$

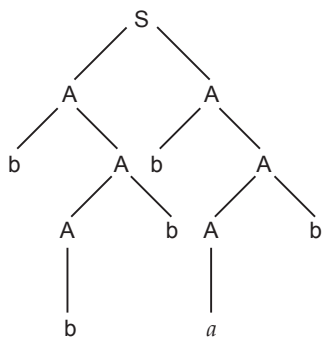
$A \rightarrow Ab \}$

Show that this is an ambiguous grammar

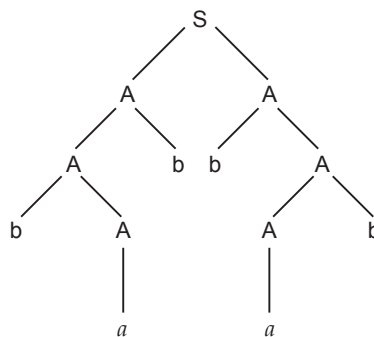
Solution : Consider a string babbab.

Refer Fig. 3.7.2.

Thus there are more than one parse tree getting generated. Hence the grammar is ambiguous.



(a)



(b)

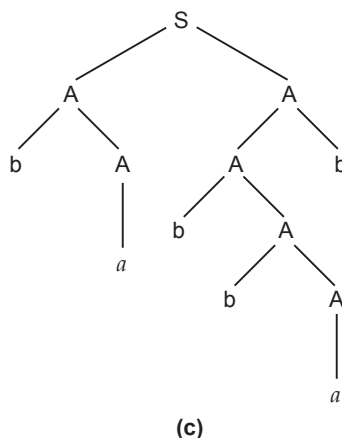


Fig. 3.7.2 For example 3.7.2

3.7.1 Inherent Ambiguity

• Definition :

A context free grammar is called inherently ambiguous if all the production rules in this grammar are ambiguous.

• Example :

For example -

$$S \rightarrow XYZ \mid aaYbb$$

$$X \rightarrow aaY \mid aa$$

$$Y \rightarrow baZ \mid ba$$

$$Z \rightarrow bZb \mid bb$$

Then the parse trees for the string "aababb" can be as shown below -

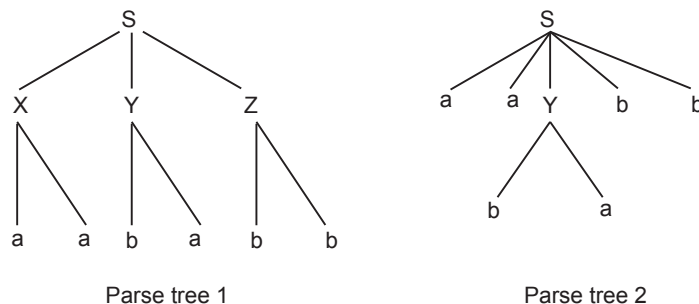


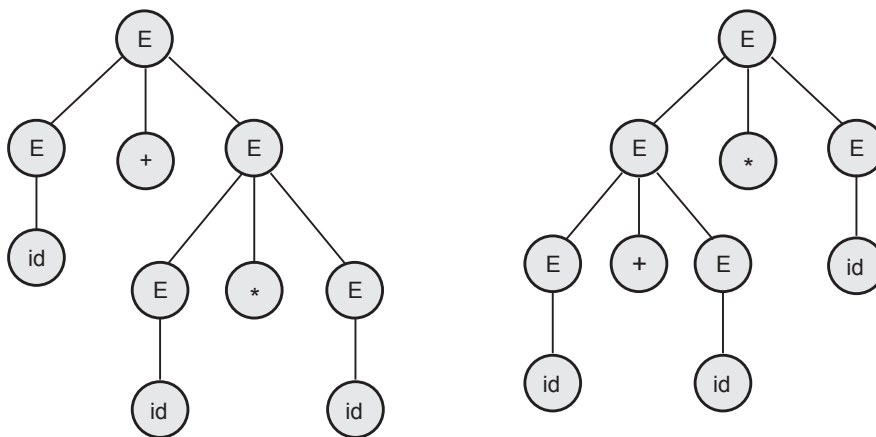
Fig. 3.7.3 Derivation trees for inherent grammar

Note that in above context free grammar there are two productions for each rule this makes every rule of the grammar as ambiguous.

3.7.2 Removal of Ambiguity

Consider the ambiguous grammar as -

$$E \rightarrow E + E \mid E * E \mid id$$



(a) Parse tree 1

(b) Parse tree 2

Fig. 3.7.4 Ambiguous grammar

is an ambiguous grammar. We will design the parse tree for $id + id * id$ as follows.

For removing the ambiguity we will apply one rule : If the grammar has left associative operator (such as $+$, $-$, $*$, $/$) then induce the left recursion and if the grammar has right associative operator (exponential operator) then induce the right recursion.

The unambiguous grammar is

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow id$$

Example 3.7.3 Give an ambiguous grammar for if then else statement and then re-write an equivalent unambiguous grammar.

SPPU : May-14, Marks 8

Solution : The ambiguous grammar for **if then else** statement is as follows -

$$S \rightarrow iCtS$$

$$S \rightarrow iCtSeS$$

$$S \rightarrow a$$

$$S \rightarrow b$$

Where **i** stands for **if**, **C** stands for **condition**, **S** stands for statement, **e** stands for **else**, and **t** stands for **then**. For making the given grammar unambiguous we match **else** with closest previous **unmatched then**. Hence on removing ambiguity we will get following rules -

$$S \rightarrow MIU$$

$$M \rightarrow iCtMeM \mid a$$

$$U \rightarrow iCtS \mid iCtMeU$$

$$C \rightarrow b$$

Here **M** and **U** non-terminals for matched and unmatched statments.

Example 3.7.4 Consider the grammar $G = (V = \{E, F\}, T = \{a, b, -\}, E, P)$

Where **P** consist of rules :

$$E \rightarrow F - E, F \rightarrow a, E \rightarrow E - F, F \rightarrow b, E \rightarrow F$$

1) Show that **G** is ambiguous

2) Remove the ambiguity, if possible of **G**.

SPPU : May-11, Marks 6

Solution :

Let,

$$E \rightarrow F - E$$

$$F \rightarrow a$$

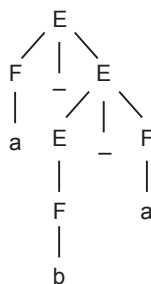
$$E \rightarrow E - F$$

$$F \rightarrow b$$

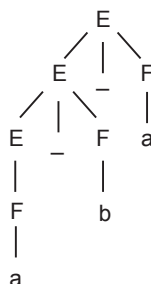
$$E \rightarrow F$$

are the production rules.

1) Consider the string **a-b-a**



Parse tree 1



Parse tree 2

2) The unambiguous grammar will be

$$E \rightarrow E - F \mid F$$

$$F \rightarrow a \mid b$$

Example 3.7.5 Given a CFG as :

$$S \rightarrow aB \mid bA$$

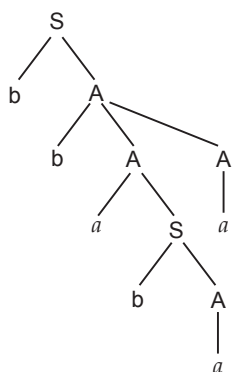
$$A \rightarrow a \mid aS \mid bAA$$

$$B \rightarrow b \mid bS \mid aBB$$

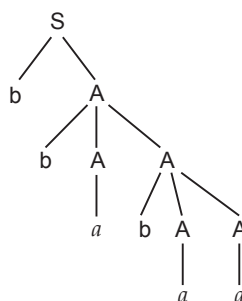
Is $ab, baba, abbbbaa$ in $L(G)$? What is the language of this CFG ? Is the CFG ambiguous ?

SPPU : May-12, Marks 8

Solution : The $\{ab, baba, abbbbaa\} \in L(G)$. The language of this CFG is L which contains equal number of a 's and b 's. The given CFG is ambiguous because consider the string $bbabaa$.



Parse tree 1



Parse tree 2

Fig. 3.7.5

Example 3.7.6 Show that following grammar is ambiguous :

$$S \rightarrow a \mid absb \mid aA b$$

$$A \rightarrow bs \mid aA A b$$

SPPU : Dec.-12, Marks 4

Solution : Consider the string $absb$. Try to find left and right derivation tree.

Left derivation tree

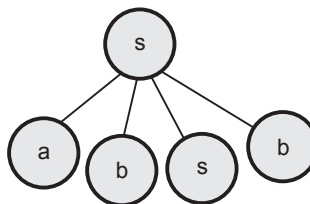
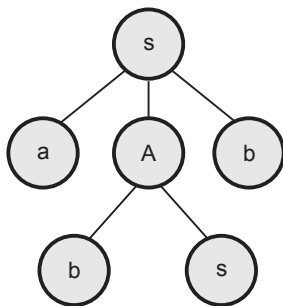


Fig. 3.7.6

For same string we have two different derivation tree so given grammar is ambiguous.

Example 3.7.7 Consider the grammar having production.

$$S \rightarrow aS \mid \epsilon$$

$$S \rightarrow aSbS$$

This grammar is ambiguous.

i) Show in particular that the string *aab* has two parse trees.

ii) Find an unambiguous grammar for the same.

SPPU : Dec.-05, Marks 8

Solution : Given grammar with production

$$S \rightarrow aS \mid \epsilon$$

$$S \rightarrow aSbS.$$

i) *a a b* has following parse trees.

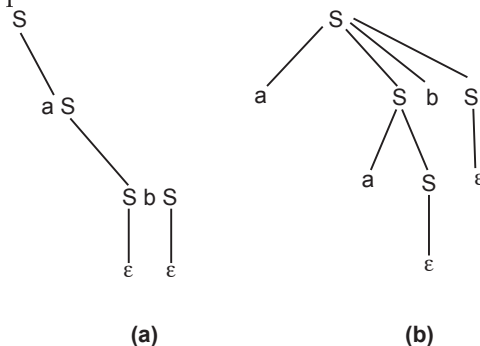


Fig. 3.7.7

unambiguous grammar for above is

$$S \rightarrow aS \mid XS \mid \epsilon$$

$$X \rightarrow aXXb \mid \epsilon$$

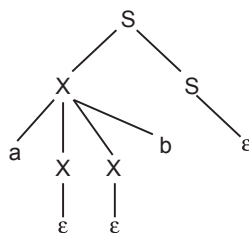


Fig. 3.7.7 (c)

Example 3.7.8 Let *G* be the grammar

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA$$

$$B \rightarrow b \mid bS \mid aBB$$

For the string *aaababbba* find

- 1) Leftmost derivation 2) Rightmost derivation
- 3) Parse tree 4) Is the grammar unambiguous

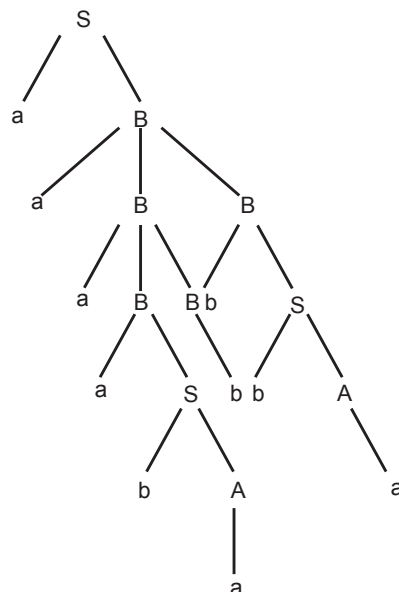
SPPU : May-07, Marks 8; Dec.-14, End Sem, Marks 4

Solution : 1) Leftmost derivation :

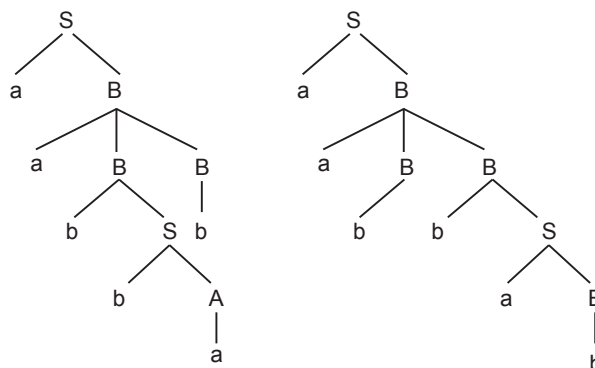
$S \rightarrow aB$
 $\rightarrow aaBB$
 $\rightarrow aaaBBB$
 $\rightarrow aaabSBB$
 $\rightarrow aaabbABB$
 $\rightarrow aaabbaBB$
 $\rightarrow aaabbababB$
 $\rightarrow aaabbabbS$
 $\rightarrow aaabbabbbA$
 $\rightarrow aaabbabbba.$

2) Rightmost derivation :

$S \rightarrow aB$
 $\rightarrow aaBB$
 $\rightarrow aaBbS$
 $\rightarrow aaBbbA$
 $\rightarrow aaBbba$
 $\rightarrow aaaBBbba$
 $\rightarrow aaaBbbba$
 $\rightarrow aaabSbbba$
 $\rightarrow aaabbAbbba$
 $\rightarrow aaabbabbba.$

3) Parse tree :**Fig. 3.7.8**

4) No, the grammar is ambiguous. Following derivations for **aabbab** shows -



Example 3.7.9 Derive $(a101 + b1) * (a1 + b)$ using leftmost and rightmost derivation where grammar is given as

$E \rightarrow I, E \rightarrow E + E, E \rightarrow E * E, E \rightarrow (E), I \rightarrow a,$

$I \rightarrow b, I \rightarrow Ia, I \rightarrow Ib, I \rightarrow I0, I \rightarrow I1.$

Test whether the grammar is ambiguous.

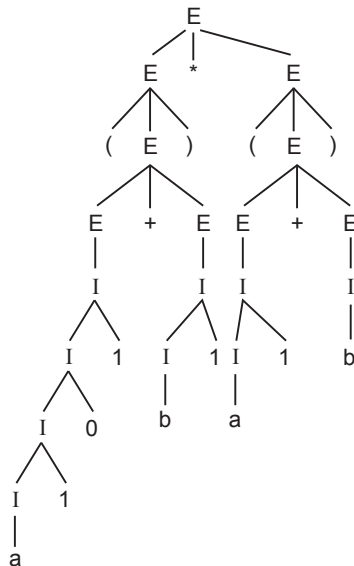
SPPU : Dec.-13, Marks 6

Solution :

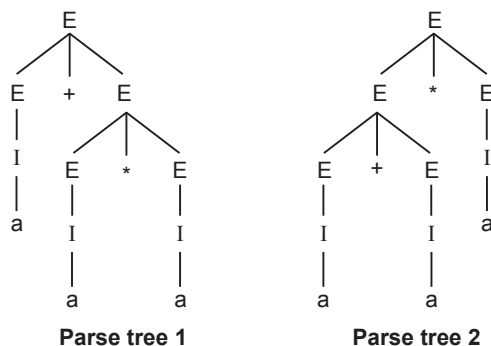
Leftmost derivation	Rightmost derivation
E	E
$E * E$	$E * E$
$(E) * E$	$E * (E)$
$(E + E) * E$	$E * (E + E)$
$(I + E) * E$	$E * (E + I)$
$(I1 + E) * E$	$E * (E + b)$
$(I01 + E) * E$	$E * (I + b)$
$(I101 + E) * E$	$E * (I1 + b)$
$(a101 + E) * E$	$E * (a1 + b)$
$(a101 + I) * E$	$(E) * (a1 + b)$
$(a101 + I1) * E$	$(E + E) * (a1 + b)$
$(a101 + b1) * E$	$(E + I) * (a1 + b)$
$(a101 + b1) * (E)$	$(E + I1) * (a1 + b)$
$(a101 + b1) * (E + E)$	$(E + b1) * (a1 + b)$

$(a101 + b1) * (I + E)$	$(I + b1) * (a1 + b)$
$(a101 + b1) * (I1 + E)$	$(I1 + b1) * (a1 + b)$
$(a101 + b1) * (a1 + E)$	$(I01 + b1) * (a1 + b)$
$(a101 + b1) * (a1 + I)$	$(I101 + b1) * (a1 + b)$
$(a101 + b1) * (a1 + b)$	$(a101 + b1) * (a1 + b)$

The derivation is as follows -



The grammar is ambiguous. Consider the derivation for the string $a + a * a$.



As there are more than one parse trees for deriving the given string the grammar is said to be ambiguous.

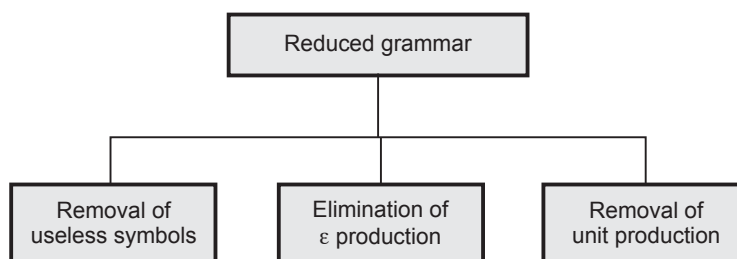
3.8 Simplification of CFG SPPU : Dec.-16, 18, 19, Aug.-17, Oct.-18, 19, Marks 8

As we have seen various languages can effectively be represented by context tree grammar. All the grammars are not always optimized. That means grammar may consists of some extra symbols (non terminals). Having extra symbols unnecessary increases the length of grammar. Simplification of grammar means reduction of grammar by removing useless symbols. The properties of reduced grammar are given below.

Properties of reduced grammar -

1. Each variable (i.e. non terminal) and each terminal of G appears in the derivation of some word in L .
2. There should not be any production as $X \rightarrow Y$ where X and Y are non terminals.
3. If ϵ is not in the language L then there need not be the production $X \rightarrow \epsilon$.

We see the reduction of grammar as below.



Let us study the reduction process in detail.

3.8.1 Removal of Useless Symbols

Any symbol is useful when it appears on the right hand side, in the production rule and generates some terminal string. If no such derivation exists then it is supposed to be an useless symbol.

A symbol p is useful if there exists some derivation in the following form.

$$S \xRightarrow{*} \alpha p \beta$$

and $\alpha p \beta \xRightarrow{*} w$

Then p is said to be useful symbol.

Where α and β may be some terminal or non terminal symbol and will help us to derive certain string w in combination with P . Let us see what exactly means the useless symbol with some example.

For example, $G = (V, T, P, S)$ where $V = \{S, T, X\}$, $T = \{0, 1\}$

$S \rightarrow 0T11T1X1011$ rule 1

$T \rightarrow 00$ rule 2

Now, in the above CFG, the non terminals are S, T and X.

To derive some string we have to start from start symbol S.

S

0T $S \rightarrow 0T$

000 $T \rightarrow 00$

Thus we can reach to certain string after following these rules.

But if $S \rightarrow X$ then there is no further rule as a definition to X. That means there is no point in the rule $S \rightarrow X$. Hence we can declare that X is a useless symbol. And we can remove this so after removal of this useless symbol CFG becomes

$G = (V, T, P, S)$ where $V = \{S, T\}$

$T = \{0, 1\}$ and $P = \{ S \rightarrow 0T11T1011$

$T \rightarrow 00 \}$

S is a start symbol.

Problems on removal of useless symbol

Example 3.8.1 Consider the CFG $G = (V, T, P, S)$

where $V = \{S, A, B\}$ $T = \{0, 1\}$

$P = \{ S \rightarrow A11B11A$

$S \rightarrow 1B11$

$A \rightarrow 0$

$B \rightarrow BB \}$ For removing useless symbols

Solution : Now in the given CFG if we try to derive any string A gives some terminal symbol as 0 but B does not give any terminal string. By following the rules with B we simply get ample number of B's and no significant string. Hence we can declare **B as useless symbol** and can remove the rules associated with it. Hence after removal of useless symbols we get,

$S \rightarrow 11A11$

$A \rightarrow 0$

Example 3.8.2 Find CFG with no useless symbols equivalent to

$S \rightarrow AB1CA$ $B \rightarrow BC1AB$

$A \rightarrow a$ $C \rightarrow aB1b$

Solution : Consider, the rule

$$S \rightarrow AB \quad 1$$

$$S \rightarrow CA \quad 2$$

In the rule 2, C and A can be replaced by some terminating string but in rule 1, B cannot be replaced by terminating string. It tends to form a never ending loop.

E.g. : $S \rightarrow AB \rightarrow aAB \rightarrow aaAB \rightarrow aaaAB$ and so on.

Thus we come to know as B is an useless symbol. Removing B the rules are now,

$$S \rightarrow CA$$

$$A \rightarrow a$$

$$C \rightarrow b$$

Example 3.8.3 Consider the following CFG

$$G = (V, \Sigma, R, S)$$

where $V = \{S, X, Y\}$

$$\Sigma = \{0, 1\}$$

$$R = \{S \rightarrow XY \mid 0$$

$$X \rightarrow 1 \}$$

Remove the useless symbols from it.

Solution : As

$$S \rightarrow XY$$

$$S \rightarrow 0$$

$$X \rightarrow 1$$

It is easy to recognize that there is no derivation for Y. Hence we can eliminate the production with Y, and the rules are

$$S \rightarrow 0$$

$$X \rightarrow 1$$

Example 3.8.4 Remove useless symbols from the following grammar.

$$S \rightarrow aA \mid bB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB$$

$$D \rightarrow ab \mid Ea$$

$$E \rightarrow aC \mid d$$

SPPU : Dec.-18, End Sem, Marks 6

Solution : We will first find all the productions which are giving terminal symbols.

$$A \rightarrow a$$

$$D \rightarrow ab$$

$$E \rightarrow d$$

Now consider start symbol

$$S \rightarrow aA \mid bB$$

Now since $B \rightarrow bB$ we will be continuously in a loop with B productions. Thus B is an useless symbol. So we will eliminate it. Then the rules are

$$\begin{array}{l} S \rightarrow aA \\ A \rightarrow a \mid aA \\ D \rightarrow ab \mid E a \\ E \rightarrow aC \mid d \end{array}$$

Again if we look at E productions there is a production $E \rightarrow aC$. But there is no rule for C. Hence we will remove the production $E \rightarrow aC$.

Now,

$$\begin{array}{l} S \rightarrow aA \\ A \rightarrow a \mid aA \\ D \rightarrow ab \mid E a \\ E \rightarrow a \end{array}$$

Now the rules S and A indicates that there is no D and E in the derivation. Again we get D and E as useless symbols. So we will eliminate D and E. Finally after removal of all useless symbols the production rules are ,

$$S \rightarrow aA$$

$$A \rightarrow aA \mid a$$

Example 3.8.5 Eliminate the useless symbols from the following grammar.

$$S \rightarrow aS \mid A \mid C$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$C \rightarrow aCb$$

Solution : We will first write all the productions that are giving terminal symbols.

$$A \rightarrow a$$

$$B \rightarrow aa$$

But if we look at start symbol S then,

$$S \rightarrow aS \mid A \mid C$$

There is no production for B from start symbol. Thus B becomes a useless symbol.

Similarly,

$$S \rightarrow C \rightarrow aCb \rightarrow aaCbb \rightarrow aaaCbbb \rightarrow \dots$$

There is no terminating symbol for C. Therefore we will eliminate C. Then set of rules are

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

Thus we obtain the grammar having these two rules after removal of useless symbols B, C.

Example 3.8.6 Eliminate the useless symbols from following grammar.

$$S \rightarrow aA \mid a \mid Bb \mid cC$$

$$A \rightarrow aB$$

$$B \rightarrow a \mid Aa$$

$$C \rightarrow cCD$$

$$D \rightarrow ddd$$

Solution : Consider all the productions that are giving terminal symbols.

$$S \rightarrow a$$

$$B \rightarrow a$$

$$D \rightarrow ddd$$

Now consider

$$S \rightarrow cC$$

$$C \rightarrow cCD$$

$$D \rightarrow ddd.$$

When we try to derive the string using production rule for C we get,

$$S \rightarrow cC \rightarrow ccCD \rightarrow ccCddd \rightarrow cccCDddd \rightarrow cccCdddddd \rightarrow \dots$$

We will not get any terminal symbol for C. Thus we get a useless symbol C. To reach to D the only rule available is by using C. But as C gets eliminated, there is no point in keeping D. Hence D also will be removed.

$$\therefore S \rightarrow aA \mid a \mid Bb$$

$$A \rightarrow aB$$

$$B \rightarrow a \mid Aa$$

is the reduced grammar.

3.8.2 Elimination of ϵ Productions from Grammar

As we have seen in finite automata and regular expression that ϵ or a null string indicates a string with no value. We have also seen that even though some NFA contains ϵ moves we can convert that NFA without ϵ moves. Even in context free grammar, if at all there is ϵ production we can remove it, without changing the meaning of the grammar. Thus ϵ productions are not necessary in a grammar.

For example,

If $S \rightarrow 0S1S1\epsilon$

Then we can remove ϵ production. But we have to take a care of meaning of CFG. i.e. meaning of CFG should not get changed if we place $S \rightarrow \epsilon$ in other rules we get $S \rightarrow 0$ when $S \rightarrow 0S$ and $S \rightarrow \epsilon$ as well as $S \rightarrow 1$ when $S \rightarrow 1S$ and $S \rightarrow \epsilon$

Hence we can rewrite the rules as

$$S \rightarrow 0S1S1011$$

Thus ϵ production is removed.

Problems on removal of ϵ production

Example 3.8.7 Remove the ϵ production from following CFG by preserving meaning of it.

$$S \rightarrow XYX$$

$$X \rightarrow 0X1\epsilon$$

$$Y \rightarrow 1Y1\epsilon$$

Solution : Now, while removing ϵ production we are deleting the rules $X \rightarrow \epsilon$ and $Y \rightarrow \epsilon$. To preserve the meaning of CFG we are actually placing ϵ at right hand side wherever X and Y have appeared.

Let us take

$$S \rightarrow XYX$$

if first X at right hand side is ϵ .

$$\text{Then } S \rightarrow YX$$

Similarly if last X in R.H.S. = ϵ .

$$\text{Then } S \rightarrow XY$$

$$\text{If } Y = \epsilon \text{ then}$$

$$S \rightarrow XX$$

If Y and X are ϵ then,

$$S \rightarrow X \text{ also}$$

$$S \rightarrow Y \text{ when both X are replaced by } \epsilon$$

$$\therefore S \rightarrow XY|YX|XX|X|Y$$

Now let us consider

$$X \rightarrow 0X$$

If we place ϵ at right hand side for X then,

$$X \rightarrow 0$$

$$X \rightarrow 0X|0$$

$$\text{Similarly } Y \rightarrow 1Y|1$$

Collectively we can rewrite the CFG with removed ϵ productions as

$$S \rightarrow XY|YX|XX|X|Y$$

$$X \rightarrow 0X|0$$

$$Y \rightarrow 1Y|1$$

Example 3.8.8 For the CFG given below remove the ϵ production

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow \epsilon$$

Solution : According to the replacement procedure, we will place ϵ at S in the sentential form.

$$S \rightarrow aSa \quad \text{if } S = \epsilon$$

$$S \rightarrow a a$$

$$\text{Similarly if } S \rightarrow bSb \quad \text{if } S = \epsilon$$

$$S \rightarrow b b$$

Thus finally the rules are

$$S \rightarrow aSa|bSb|aa|bb$$

Example 3.8.9 Eliminate the ϵ productions from the CFG given below.

$$A \rightarrow 0B1|1B1$$

$$B \rightarrow 0B|1B|\epsilon$$

Solution : Now the ϵ production is $B \rightarrow \epsilon$ we will delete this production. And then add the productions having B replaced by ϵ .

$A \rightarrow 0 B 1$
 if $B = \epsilon$
 $A \rightarrow 0 1$
 Similarly $A \rightarrow 1 B 1 \rightarrow 11$
 $\therefore A \rightarrow 0 B 1 | 1 B 1 | 01 | 11$
 Similarly, $B \rightarrow 0 B$
 if $B = \epsilon$
 $B \rightarrow 0$
 as well as $B \rightarrow 1$
 $B \rightarrow 0 B | 1 B | 0 | 1$

Collectively, we can write

$A \rightarrow 0 B 1 | 1 B 1 | 01 | 11$
 $B \rightarrow 0 B | 1 B | 0 | 1$

Example 3.8.10 Construct CFG without ϵ production from the one which is given below.

$S \rightarrow a | A b | a B a$
 $A \rightarrow b | \epsilon$
 $B \rightarrow b | A$

Solution : If you observe carefully, then not only A have ϵ production but even B also indicates ϵ production i.e. $A \rightarrow \epsilon$ straight forward but $B \rightarrow A \rightarrow \epsilon$. Let us apply the method of replacement.

$S \rightarrow A b$
 if $A = \epsilon$ then
 $S \rightarrow b$
 if $B = \epsilon$
 $S \rightarrow a a$
 $S \rightarrow a | A b | b | a a | a B a$
 $A \rightarrow b$
 $B \rightarrow b$

Finally the rules are

$S \rightarrow a | A b | b | a a | a B a$
 $A \rightarrow b$
 $B \rightarrow b$

Example 3.8.11 Consider the CFG given below.

$$S \rightarrow Y$$

$$X \rightarrow Zb$$

$$Y \rightarrow BW$$

$$Z \rightarrow AB$$

$$W \rightarrow Z$$

$$A \rightarrow aA \mid bA \mid \epsilon$$

$$B \rightarrow Ba \mid Bb \mid \epsilon$$

Solution : In this example only A and B shows ϵ productions.

If we put $A = \epsilon$ in A production then

$$A \rightarrow a \mid b$$

$$B \rightarrow a \mid b$$

Also

$$Z \rightarrow AB \quad \text{Putting } A = \epsilon, B = \epsilon$$

$$Z \rightarrow \epsilon\epsilon \rightarrow \epsilon$$

So wherever Z appears we place ϵ

$$X \rightarrow b$$

Since $W \rightarrow Z \rightarrow \epsilon$ we can straightaway delete it.

$$\therefore Y \rightarrow b$$

Collectively let us rewrite the rules

$$S \rightarrow XY$$

$$X \rightarrow b$$

$$Y \rightarrow b$$

$$A \rightarrow aA \mid bA \mid a \mid b$$

$$B \rightarrow Ba \mid Bb \mid a \mid b$$

Since S is a start symbol and it defines XY only whereas X and Y yields b as a terminal symbol. There is no chance for producing rules by A or B. Hence A and B are useless symbols and we can remove it. Finally rules are

$$S \rightarrow XY$$

$$X \rightarrow b$$

$$Y \rightarrow b$$

which ultimately

$$S \rightarrow bb$$

Example 3.8.12 Consider the CFG given below,

For eliminating ϵ productions

$$S \rightarrow P 0 Q \mid Q Q \mid 0 R \mid R Q P$$

$$P \rightarrow R 0 \mid 1 R \mid R R \mid R Q P$$

$$Q \rightarrow Q 0 \mid P Q \mid \epsilon$$

$$R \rightarrow 0 P \mid Q Q Q$$

Solution : As we can see that $Q \rightarrow \epsilon$.

$$\text{Similarly} \quad R \rightarrow QQQ \rightarrow \epsilon.$$

Hence we will replace Q and R by ϵ and accordingly add the production rules.

$$\text{Similarly} \quad P \rightarrow R R \rightarrow \epsilon$$

Let us modify the rules if P or Q or R becomes ϵ .

$$S \rightarrow P 0 Q \mid 0 0 R \mid Q Q \mid R Q P$$

$$P \rightarrow R 0 \mid 1 1 R \mid R R \mid R Q P$$

$$Q \rightarrow Q 0 \mid P Q$$

$$R \rightarrow 0 P \mid Q Q Q$$

Note that with P, Q, R other meanings are associated, they are not purely giving ϵ to us. Therefore we cannot remove P, Q and R symbols.

Example 3.8.13 Eliminate ϵ - productions from the grammar G

$$A \rightarrow aBb \mid bBa$$

$$B \rightarrow aB \mid bB \mid \epsilon$$

SPPU : Aug.-17, In Sem, Marks 3

Solution : $B \rightarrow \epsilon$ is used at RHS of each production rule. On eliminating ϵ production we get

$$A \rightarrow aBb \mid bBa \mid ab \mid ba$$

$$B \rightarrow aB \mid bB \mid a \mid b$$

3.8.3 Removing Unit Productions

The unit productions are the productions in which one non terminal gives another non terminal.

For example if

$$X \rightarrow Y$$

$$Y \rightarrow Z \quad Z \rightarrow X$$

Then X , Y and Z are unit productions. To optimize the grammar we need to remove the unit productions.

If $A \xRightarrow{*} B$ is a unit production and $B \rightarrow X_1 X_2 X_3 \dots X_n$ then while removing $A \rightarrow B$ production we should add a rule

$$\overset{\bullet}{A} \rightarrow X_1 X_2 X_3 \dots X_n.$$

Let us solve something.

Example 3.8.14 If the CFG is as below.

$$S \rightarrow 0A \mid 1B \mid C$$

$$A \rightarrow 0S \mid 00$$

$$B \rightarrow 1 \mid A$$

$$C \rightarrow 01$$

then remove the unit productions.

Solution : Clearly $S \rightarrow C$ is a unit production. But while removing $S \rightarrow C$ we have to consider what C gives. So, we can add a rule to S .

$$S \rightarrow 0A \mid 1B \mid 01$$

Similarly $B \rightarrow A$ is also a unit production so we can modify it as

$$B \rightarrow 1 \mid 0S \mid 00$$

Thus finally we can write CFG without unit production as

$$S \rightarrow 0A \mid 1B \mid 01$$

$$A \rightarrow 0S \mid 00$$

$$B \rightarrow 1 \mid 0S \mid 00$$

$$C \rightarrow 01$$

Example 3.8.15 Optimize the CFG given below by reducing the grammar.

S is a start symbol.

$$S \rightarrow A \mid 0C1$$

$$A \rightarrow B \mid 01 \mid 10$$

$$C \rightarrow \epsilon \mid CD$$

Solution : $S \rightarrow A \rightarrow B$ is a unit production

$C \rightarrow \epsilon$ is a null production.

$C \rightarrow CD$ B and D are useless symbol.

Reducing a grammar we have to avoid all the above conditions.

Let $S \rightarrow A$

i.e. $A \rightarrow B$ is a useless symbol because B is not defined further more.

$$S \rightarrow 01110$$

i.e. $S \rightarrow 0111010C1$

But $C \rightarrow \varepsilon$

Hence ultimately $S \rightarrow 01110$

$A \rightarrow B$ but we can remove this production since B is a useless symbol.

Hence $A \rightarrow 01110$

But the start symbol $S \rightarrow 01110$

There is no A in the derivation of A so by considering A also as a useless symbol we get final CFG as

$$S \rightarrow 01110$$

Example 3.8.16 Eliminate the unit productions from following grammar

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow C|b$$

$$C \rightarrow D$$

$$D \rightarrow E|bC$$

$$E \rightarrow d|Ab$$

Solution : As,

$$S \rightarrow AB \text{ and}$$

$$A \rightarrow a$$

There is only one rule with A and that too giving terminal symbol. Hence there is no question of getting unit production with A. Now consider

$$B \rightarrow C$$

$$C \rightarrow D$$

$$D \rightarrow E$$

It is clear that B, C and D are unit productions. As $E \rightarrow d|Ab$, we will replace value of D. Then,

$$D \rightarrow d|Ab|bC$$

Similarly as $C \rightarrow D$ we can write

$$C \rightarrow d|Ab|bC$$

Hence B becomes

$$B \rightarrow d|Ab|bC|b$$

Thus the grammar after removing unit productions

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow d|Ab|bC|b$$

$$C \rightarrow d|Ab|bC$$

$$D \rightarrow d|Ab|bC$$

$$E \rightarrow d|Ab.$$

Now there is no path for D and E. From start state we can remove them by considering useless symbols. The optimized grammar will be

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow d|Ab|bC|b$$

$$C \rightarrow d|Ab|bC$$

Example 3.8.17 Simplify the following grammar :

$$i) S \rightarrow Ab, A \rightarrow a, B \rightarrow C | b, C \rightarrow D, D \rightarrow E, E \rightarrow a$$

$$ii) S \rightarrow 0A0 | 1B1 | BB, A \rightarrow C, B \rightarrow S | A, C \rightarrow S | \epsilon$$

SPPU : Dec.-16, End Sem, Marks 8, Oct.-19, In Sem, Marks 4

Solution :

i) Let,

$$S \rightarrow Ab$$

$$A \rightarrow a$$

$$B \rightarrow C | b$$

$$C \rightarrow D$$

$$D \rightarrow E$$

$$E \rightarrow a$$

Step 1 : Here $B \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$ and $E \rightarrow a$, are unit productions. These can be eliminated and we get simply $B \rightarrow a | b$.

Step 2 : Now if we observe the start state S, we can clearly say that non terminal B is non-reachable. Hence eliminate it as useless symbol.

Step 3 : Thus we get the simplified grammar, on removing B, C, D and E as

$$S \rightarrow Ab$$

$$A \rightarrow a$$

ii) Let,

$$S \rightarrow 0A0$$

$$S \rightarrow 1B1$$

$$S \rightarrow BB$$

$$A \rightarrow C$$

$$B \rightarrow S \mid A$$

$$C \rightarrow S \mid \epsilon$$

Step 1 : The $A \rightarrow C$, $C \rightarrow S \mid \epsilon$, are unit productions. These can be eliminated
 $A \rightarrow S \mid \epsilon$.

Step 2 :

$B \rightarrow A$, $A \rightarrow S \mid \epsilon$ are unit productions, these can be eliminated by $B \rightarrow \epsilon$

Step 3 : As we have

$$S \rightarrow 0A0 \mid 1B1$$

Also $A \rightarrow S$ and $B \rightarrow S$

$\therefore$ We can reduce it as

$$S \rightarrow 0S0 \mid 1S1$$

Step 4 : Similarly $S \rightarrow BB$ can be reduced.

Step 5 : Finally we get

$$S \rightarrow 0S0 \mid 1S1 \mid \epsilon$$

as simplified productions.

Example 3.8.18 Consider the grammar $G = \{(A,B), (a,b), P, A\}$ where P consists of

$$A \rightarrow B$$

$$B \rightarrow a \mid b$$

Eliminate unit productions.

SPPU : Oct.-18, In Sem, Marks 3

Solution : $A \rightarrow B$ is a unit production. We eliminate it and rewrite the production rule as - $A \rightarrow a \mid b$.

Example 3.8.19 Optimize the CFG given below by reducing the grammar where S is a start symbol.

$$S \rightarrow A \mid 0CI$$

$$A \rightarrow B \mid 0I \mid 10$$

$$C \rightarrow \epsilon \mid CD$$

SPPU : Oct.-19, In Sem, Marks 4

Solution :

- 1) In given production rules, B and D as useless symbol,
- 2) $S \rightarrow A \rightarrow B$ is unit production
- 3) $C \rightarrow \epsilon$ is null production
- 4) To optimize given production, we must eliminate useless symbol, unit and null production. Hence optimized production is $S \rightarrow 0I \mid 10$

Example 3.8.20 Simplify the following Context Free Grammar (CFG) :

$$S \rightarrow ASB \mid \epsilon$$

$$A \rightarrow aAS \mid \epsilon$$

$$B \rightarrow SbS \mid A \mid bb$$

$G = \{(S, A, B), (a, b), S, P : \text{Productions are given above.}$ **SPPU : Dec.-19, End Sem, Marks 6**

Solution : Let,

$$S \rightarrow ASB \mid \epsilon$$

$$A \rightarrow aAS \mid \epsilon$$

$$B \rightarrow SbS \mid A \mid bb$$

We will eliminate unit productions

$$S \rightarrow A$$

Hence, $S \rightarrow ASB \mid \epsilon$

$$A \rightarrow aAS \mid \epsilon$$

$$B \rightarrow SbS \mid aAS \mid bb \mid \epsilon$$

Now we will eliminate ϵ productions

Hence, $S \rightarrow ASB \mid AB \mid AS \mid SB$

$$A \rightarrow aAS \mid aA \mid a$$

$$B \rightarrow SbS \mid aAS \mid bb \mid Sb \mid bS \mid b \mid aA \mid aS \mid bb$$

3.9 Normal Forms

**SPPU : Dec.-05, 10, 13, 14, 15, 16, 17, 19,
May-08, 10, 13, 14, 18, Oct.-18, Marks 12**

There are two important normal forms - **Chomsky's Normal Form** and **Greibach Normal Form**

Let us study these normal forms with the help of examples -

3.9.1 Chomsky Normal Form

The Chomsky's Normal Form can be defined as

Non terminal $\rightarrow$ Non terminal $\cdot$ Non terminal
Non terminal $\rightarrow$ Terminal

The given CFG should be converted in the above format then we can say that the grammar is in CNF. Before converting the grammar into CNF it should be in reduced form. That means remove all the useless symbols, ϵ productions and unit productions from it. Thus this reduced grammar can be then converted to CNF.

Let us solve some examples for Chomsky's Normal Form.

Problems on CNF

Example 3.9.1 Convert the following CFG into CNF

$S \rightarrow aaaaS$

$S \rightarrow aaaa$

Solution : As we know the rule for Chomsky's normal form is

Non terminal $\rightarrow$ Non terminal $\cdot$ Non terminal

Non terminal $\rightarrow$ Terminal

But the CFG given is

$S \rightarrow aaaaS$ rule 1

$S \rightarrow aaaa$ rule 2

If we add a rule

Then our rule 1 and 2 becomes,

$A \rightarrow AAAAS$

$S \rightarrow AAAA$

Let us take

$S \rightarrow A$ AAAS can be replaced by P_1

If we define

$P_1 \rightarrow AAAS$ then the rule becomes

$S \rightarrow A P_1$ which is in CNF

But the new rule P_1 is not CNF, so let us convert it

$P_1 \rightarrow A \boxed{AAS}$ can be replaced by P_2

then $P_1 \rightarrow A P_2$ which is in CNF

and $P_2 \rightarrow AAS$ which is not in CNF so let us convert it to CNF

$P_2 \rightarrow A \boxed{AS}$ can be replaced by P_3

$P_2 \rightarrow A P_3$ where $P_3 \rightarrow AS$

Now both P_2 and P_3 are in CNF.

Collectively rewrite these rules (these all indicate rule 1 of the given CFG).

$$\begin{array}{l} S \rightarrow A P_1 \\ P_1 \rightarrow A P_2 \\ P_2 \rightarrow A P_3 \\ P_3 \rightarrow A S \end{array}$$

Now consider rule 2

$S \rightarrow AAAA$

If we break the rule 2 as

$$\begin{array}{ccc} S \rightarrow & \boxed{AA} & \boxed{AA} \\ & \downarrow & \downarrow \\ & P_4 & P_5 \end{array} \quad \text{indicated by}$$

The rule becomes

$S \rightarrow P_4 P_5$ which is in CNF.

But P_4 and P_5 indicate the same rule. So we can eliminate either of them.

Hence rule 2 becomes

$S \rightarrow P_4 P_4$

Finally we can collectively show the CFG converted to CNF as

$S \rightarrow A P_1$

$$P_1 \rightarrow A P_2$$

$$P_2 \rightarrow A P_3$$

$$P_3 \rightarrow A S$$

$$S \rightarrow P_4 P_4$$

$$P_4 \rightarrow AA$$

$$A \rightarrow a$$

Example 3.9.2 Convert the given CFG to CNF $S \rightarrow aSa \mid bSb \mid a \mid b$

Solution : Let us start by adding new symbols for the terminals.

$$S \rightarrow ASA$$

$$A \rightarrow a$$

$$S \rightarrow BSB$$

$$B \rightarrow b$$

Note that although $s \rightarrow a$ and $A \rightarrow a$ are similar still we are not replacing any a by S . This is because, S is not simply giving a . It has other productions also. We want such a non terminal having only one production and that is a . Same is with B . Hence we have added the rules $A \rightarrow a$ and $B \rightarrow b$.

Let us take $S \rightarrow ASA$ for converting to CNF.

$$S \rightarrow A \boxed{SA}$$

replace it by A_1

$$S \rightarrow A A_1$$

$$A_1 \rightarrow SA$$

Take, $S \rightarrow B \boxed{SB}$

replace it by A_2

$$S \rightarrow B A_2 \quad \text{both are in CNF}$$

$$A_2 \rightarrow SB$$

Hence we can write the rule in CNF as

$$S \rightarrow A A_1$$

$$A_1 \rightarrow SA$$

$$A \rightarrow a$$

$$S \rightarrow B A_2$$

$$A_2 \rightarrow SB$$

$$B \rightarrow b$$

$$S \rightarrow a$$

$$S \rightarrow b$$

Example 3.9.3 Consider $a = (\{S, A\}, \{a, b\}, P, S)$

Where P consists of

$$S \rightarrow aAS|a$$

$$A \rightarrow SbA|SS|ba$$

Convert it to its equivalent CNF.

Solution : Let us consider,

$$S \rightarrow aAS$$

If we put $R_1 \rightarrow a$

then $S \rightarrow R_1AS$

and if $R_2 \rightarrow AS$

then S becomes

$$S \rightarrow R_1 R_2 \text{ is now in CNF}$$

$$S \rightarrow a \text{ is already in CNF}$$

$$A \rightarrow SbA$$

Let $R_3 \rightarrow b$

Hence $A \rightarrow S R_3A$

Add $R_4 \rightarrow R_3A$

then $A \rightarrow S R_4$

$$A \rightarrow SS \text{ is already in CNF}$$

Now as we have defined $R_1 \rightarrow a$ and $R_3 \rightarrow b$.

$A \rightarrow R_1 R_3$ will be a conversion in CNF of $A \rightarrow ab$.

Finally, we can write

$$S \rightarrow R_1 R_2 \quad A \rightarrow S R_4$$

$$S \rightarrow a \quad R_4 \rightarrow R_3A$$

$$R_1 \rightarrow a \quad R_3 \rightarrow b$$

$$R_2 \rightarrow AS \quad A \rightarrow SS$$

$$A \rightarrow R_1 R_3$$

Example 3.9.4 Convert the given CFG to CNF.

Consider $G = (V, T, P, S)$

Where $V = \{S, A, B\}$

$T = \{a, b\}$

P consists of

$S \rightarrow aB$ $A \rightarrow bAA$

$S \rightarrow bA$ $B \rightarrow b$

$A \rightarrow a$ $B \rightarrow bS$

$A \rightarrow aS$ $B \rightarrow aBB$

**SPPU : Dec.-05, Marks 12; Dec.-14, In Sem, Marks 6;
Dec.-16, End Sem, Marks 8**

Solution : Let us start with first rule.

$R_1 \rightarrow a$

$S \rightarrow R_1B$ is in CNF for $S \rightarrow aB$

$R_2 \rightarrow b$

$S \rightarrow R_2A$ is in CNF for $S \rightarrow bA$

$A \rightarrow a$

$A \rightarrow aS$ can be written as

$A \rightarrow R_1S$

Now $A \rightarrow bAA$ can be written as

$A \rightarrow R_2 \boxed{AA}$ replace it by R_3

i.e. $R_3 \rightarrow AA$

then $A \rightarrow R_2R_3$

$B \rightarrow a$

$B \rightarrow bS$ can be written as

$B \rightarrow R_2S$ is in CNF

Now, $B \rightarrow aBB$

i.e. $B \rightarrow R_1BB$ $\because R_1 \rightarrow a$

Let $R_4 \rightarrow BB$

then $B \rightarrow R_1 R_4$

Finally we can write,

$S \rightarrow R_1B$

$$\begin{aligned}
S &\rightarrow R_2 A \\
A &\rightarrow R_1 S \\
A &\rightarrow R_2 R_3 \\
B &\rightarrow b \\
B &\rightarrow R_2 S \\
B &\rightarrow R_1 R_4 \\
R_1 &\rightarrow a \\
R_2 &\rightarrow b \\
R_3 &\rightarrow AA \\
R_4 &\rightarrow BB
\end{aligned}$$

is a Chomsky's normal form.

Example 3.9.5 Convert the following CFG to CNF

$$\begin{aligned}
S &\rightarrow ABA \\
A &\rightarrow aA \mid \epsilon \\
B &\rightarrow bB \mid \epsilon
\end{aligned}$$

SPPU : May-13, 14, Marks 6

Solution : In the Chomsky's normal form the ϵ production is not allowed. So first we will eliminate ϵ productions.

$$A \rightarrow \epsilon \quad \text{and} \quad B \rightarrow \epsilon$$

If put ϵ instead of A and B

We can get,

$$S \rightarrow ABA \mid AB \mid BA \mid AA \mid A \mid B$$

Similarly, $A \rightarrow aA \mid a$

$$B \rightarrow bB \mid b$$

Now $S \rightarrow A$ and $S \rightarrow B$ is unit production getting introduced in the grammar after removal of ϵ production. So we will remove unit production also,

$$S \rightarrow ABA \mid AB \mid BA \mid AA \mid aA \mid a \mid bB \mid b$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Now let us convert this grammar to Chomsky's normal form.

Let

$$S \rightarrow ABA$$

$$\boxed{S \rightarrow ABA}$$

$$\boxed{R_1 \rightarrow BA}$$

then $S \rightarrow AR_1$

Now $\boxed{S \rightarrow AB} \quad \boxed{S \rightarrow BA} \quad \boxed{S \rightarrow AA}$ already in CNF

Let $S \rightarrow aA$

We will define $R_2 \rightarrow a$

$\therefore$

$$\boxed{S \rightarrow R_2A}$$

$$\boxed{R_2 \rightarrow a}$$

$$\boxed{S \rightarrow a}$$

already in CNF

$$S \rightarrow bB$$

Let

$$\boxed{R_3 \rightarrow b}$$

then

$$\boxed{S \rightarrow R_3B}$$

$$S \rightarrow b \text{ already in CNF}$$

Now consider the rules,

$$\boxed{A \rightarrow aA}$$

We can rewrite it as

$$\boxed{A \rightarrow R_2A}$$

$$\boxed{A \rightarrow a}$$

Also, $B \rightarrow bB$ can be rewritten as

$$B \rightarrow R_3 B$$

$$B \rightarrow b$$

Collect all the CNF s to integrate and we will get

$$S \rightarrow AR_1 | AB | BA | AA | R_2A | R_3B | a | b$$

$$A \rightarrow R_2A | a$$

$$B \rightarrow R_3B | b$$

$$R_1 \rightarrow BA$$

$$R_2 \rightarrow a$$

$$R_3 \rightarrow b$$

is done.

Example 3.9.6 Convert the following grammar to Chomsky's normal form

$$S \rightarrow a \mid b \mid a S S.$$

Solution : The Chomsky's normal form is

$$NT \rightarrow NT \cdot NT$$

$$NT \rightarrow \text{Terminal}$$

We will convert the given grammar in this form

$$S \rightarrow a \quad \text{already in CNF}$$

$$S \rightarrow b \quad \text{already in CNF}$$

$$S \rightarrow a S S$$

Now we will convert

$$S \rightarrow a S S \quad \text{into equivalent CNF}$$

$$S \rightarrow R_1 R_2$$

$$R_1 \rightarrow a$$

$$R_2 \rightarrow S S$$

$\therefore$ The equivalent CNF of given grammar is -

$$S \rightarrow a$$

$$S \rightarrow b$$

$$S \rightarrow R_1 R_2$$

$$R_1 \rightarrow a$$

$$R_2 \rightarrow S S.$$

Example 3.9.7 Convert the following grammar to Chomsky's normal form

$$S \rightarrow \sim S \mid [S \supset S] \mid p \mid q.$$

SPPU : Dec.-13, Marks 6

Solution : Chomsky's normal form is -

Non terminal $\rightarrow$ Non terminal $\cdot$ Non terminal

Non terminal $\rightarrow$ Terminal

Consider given grammar rule by rule and let us convert it into CNF

$$S \rightarrow \sim S$$

Can be written as

$$S \rightarrow AS$$

$$A \rightarrow \sim$$

Similarly $S \rightarrow [S \supset S]$

Can be written as

$$S \rightarrow BC$$

$$B \rightarrow DE$$

$$E \rightarrow SF$$

$$D \rightarrow [$$

$$F \rightarrow \supset$$

$$C \rightarrow SG$$

$$G \rightarrow]$$

And, $S \rightarrow p$

$$S \rightarrow q$$

is already in CNF.

Hence the complete grammar written in CNF is -

$$S \rightarrow AS$$

$$A \rightarrow \sim$$

$$S \rightarrow BC$$

$$B \rightarrow DE$$

$$E \rightarrow SF$$

$$D \rightarrow [$$

$$F \rightarrow \supset$$

$$C \rightarrow SG$$

$$G \rightarrow]$$

$$S \rightarrow p$$

$$S \rightarrow q$$

Example 3.9.8 Convert the following grammar to CNF

$$S \rightarrow AACD$$

$$A \rightarrow aAb \mid \epsilon$$

$$C \rightarrow aC \mid a$$

$$A \rightarrow aDa \mid bDb \mid \epsilon$$

SPPU : May-10, Dec.-10, Marks 6, Oct.-18, In Sem, Marks 4

Solution : Before converting any grammar to its normal form we must first simplify it by removing ϵ production and by eliminating useless symbols.

Let $S \rightarrow A A A D$

But there is no production rule defined for given non-terminal D. Hence S and D are useless symbols. But S is a start symbol and if we eliminate it then the grammar becomes **invalid**. Hence **conversion** of such grammar to CNF is **not possible**.

Example 3.9.9 Reduce the following grammar to Chomsky Normal Form :

$$S \rightarrow 1A \mid 0B$$

$$A \rightarrow 1AA \mid 0S \mid 0$$

$$B \rightarrow 0BB \mid 1S \mid 1$$

SPPU : Dec.-13, Marks 6

Solution : Refer example 3.9.4 by assuming $a = 0$ and $b = 1$.

Example 3.9.10 Convert the following CFG into Chomsky Normal Form (CNF) :

$$S \rightarrow AB$$

$$A \rightarrow CA \mid \wedge$$

$$B \rightarrow DB \mid \wedge$$

$$C \rightarrow 011 \mid 1$$

$$D \rightarrow 01$$

SPPU : Dec.-15, End Sem, Marks 6

Solution : Before converting the given grammar to CNF we will eliminate $\wedge$ production.

$A \rightarrow CA \mid \wedge$ becomes

$A \rightarrow CA \mid 011 \mid 1$

Similarly

$B \rightarrow DB \mid \wedge$ becomes

$B \rightarrow DB \mid 01$

The CNF conversion is as follows :

Production Rule	CNF
$S \rightarrow AB$	already is in CNF
$A \rightarrow CA$	is already in CNF

$A \rightarrow 011$	$A \rightarrow XY$ $X \rightarrow 0$ $Y \rightarrow ZZ$ $Z \rightarrow 1$
$B \rightarrow DB$	is already in CNF
$B \rightarrow XZ$	

3.9.2 Greibach Normal Form

Now we will discuss one more interesting normal form called **Greibach Normal Form**.

The rule for GNF is

Non - terminal $\rightarrow$ One terminal. Any number of non-terminals

In short,

$$NT \rightarrow t. NT$$

For example :

$$\left. \begin{array}{l} S \rightarrow aA \\ S \rightarrow a \end{array} \right\} \text{ is in GNF}$$

But $\left. \begin{array}{l} S \rightarrow AA \\ \text{Or } S \rightarrow Aa \end{array} \right\} \text{ is not in GNF}$

To convert given CFG into GNF very interesting procedure is followed.

Step 1 : Convert the given grammar into simplified form by eliminating ϵ productions, unit productions and useless symbols.

Step 2 : Bring the grammar to Chomsky's Normal Form (CNF). This step can be optional.

Step 3 : Number the non-terminal symbols in ascending order i.e. A_1, A_2, A_3 and so on.

Step 4 : In the rule of the form

$$A_i \rightarrow A_j A_k \dots$$

There must be $i < j$. If $i > j$ or $i = j$ then process the rule to convert it to GNF.

When $i = j$ i.e.

$$A_i \rightarrow A_j A_k \dots$$

then that grammar is said to be **left recursive**. Remove the left recursion using following formula

$$\text{If } A \rightarrow A\alpha|\beta$$

then convert above grammar in the following form

$$A \rightarrow \beta A'|\beta$$

$$A' \rightarrow \alpha A'|\alpha$$

Step 5 : Then convert each rule in the form

Non-terminal $\rightarrow$ one terminal $\cdot$ Any number of non-terminals

Let us solve some problems based on conversion to GNF.

Example 3.9.11 Convert the following grammar to Greibach Normal Form

$$S \rightarrow abSb$$

$$S \rightarrow aa$$

Solution : The Greibach Normal Form is

$$NT \rightarrow t \text{ (set of NT)}$$

Step 1 : There is no ϵ , or unit production. So there is n need to simplify the grammar.

Step 2 : Convert the grammar to CNF.

$S \rightarrow abSb$	$S \rightarrow XY$ $X \rightarrow AB$ $Y \rightarrow SB$ $A \rightarrow a$ $B \rightarrow b$
$S \rightarrow aa$	$S \rightarrow AA$

Step 3 : Now rename non-terminals in ascending order.

$S \rightarrow XY$ can be converted to $A_1 \rightarrow A_2 A_3$

$$X \rightarrow AB \Rightarrow A_2 \rightarrow A_4 A_5$$

$$Y \rightarrow SB \Rightarrow A_3 \rightarrow A_1 A_5$$

$$A \rightarrow a \Rightarrow A_4 \rightarrow a$$

$$B \rightarrow b \Rightarrow A_5 \rightarrow b$$

Step 4 : i) Now we will verify the condition of A_i, A_j as if $A_i < A_j$?

The rules are

$$A_1 \rightarrow A_2 A_3$$

$$A_2 \rightarrow A_4 A_5$$

$$\boxed{A_3 \rightarrow A_1 A_5} \rightarrow \begin{array}{c} \textcircled{A_3} \\ \uparrow \\ A_i \end{array} \rightarrow \begin{array}{c} \textcircled{A_1} \\ \uparrow \\ A_j \end{array} A_5 \rightarrow \text{Here } i > j \\ \text{i.e. } 3 > 1$$

$$A_4 \rightarrow a$$

$$A_5 \rightarrow b$$

Therefore we will replace A_1 on R.H.S

$$\begin{array}{l} A_3 \rightarrow \boxed{A_1} A_5 \\ \downarrow \\ A_3 \rightarrow \boxed{A_2} A_3 A_5 \quad \because A_1 \rightarrow A_2 A_3 \\ \downarrow \\ A_3 \rightarrow A_4 A_5 A_3 A_5 \end{array}$$

Now we get $i < j$ to be satisfied for $A_i \rightarrow A_j A_k \dots$

To summarize,

- 1) $A_1 \rightarrow A_2 A_3$
- 2) $A_2 \rightarrow A_4 A_5$
- 3) $A_3 \rightarrow A_4 A_5 A_3 A_5$
- 4) $A_4 \rightarrow a$
- 5) $A_5 \rightarrow b$

ii) Now consider rules for non terminals A_2 and A_3 . The A_4 on RHS is replaced by terminal symbol **a**. The rules then become

- 1) $A_1 \rightarrow A_2 A_3$
- 2) $A_2 \rightarrow a A_5$
- 3) $A_3 \rightarrow a A_5 A_3 A_5$
- 4) $A_4 \rightarrow a$
- 5) $A_5 \rightarrow b$

iii) If we observe all the above rules we find that all the rules are in GNF except rule (1). So we will consider it.

$$A_1 \rightarrow A_2 A_3$$

Replace A_2 by its RHS.

$$\therefore A_1 \rightarrow a A_5 A_3$$

To summerise all, we get following rules that are all in GNF.

$$A_1 \rightarrow aA_5A_3$$

$$A_2 \rightarrow aA_5$$

$$A_3 \rightarrow aA_5A_3A_5$$

$$A_4 \rightarrow a$$

$$A_5 \rightarrow b$$

Example 3.9.12 Convert the given CNF to CFG

$$S \rightarrow ABA$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

Solution :

Step 1 : • There is ϵ - Production, so we will simplify the given grammar as follows

$$S \rightarrow ABA \mid BA \mid AB \mid AA \mid A \mid B$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

• Now eliminate unit productions

$$S \rightarrow A \mid B$$

so,

$$S \rightarrow ABA \mid BA \mid AB \mid AA \mid aA \mid a \mid bB \mid b$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Step 2 :

- In above grammar the rules for A and B non-terminals are already in GNF.
- In fact, some of the rules of S are also in GNF.
- Hence we will handle,

$$S \rightarrow ABA \mid BA \mid AB \mid AA$$

by simply substituting the values of A and B. Here we need not have to bring grammar in ascending order of $A_i, A_j, A_k \dots$

Step 3 : We will then get -

$S \rightarrow ABA$	$S \rightarrow aA BA \mid aBA$
$S \rightarrow BA$	$S \rightarrow bBA \mid bA$
$S \rightarrow AB$	$S \rightarrow aAB \mid aB$
$S \rightarrow AA$	$S \rightarrow aAA \mid aA$

Step 4 : Thus now all the rules are in GNF. To summerize

$$S \rightarrow aABA \mid aBA \mid bBA \mid bA \mid aAB \mid aB \mid aAA \mid aA$$

$$S \rightarrow aA \mid a \mid bB \mid b$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Example 3.9.13 Convert given CFG to GNF where $V = \{S, A\}$, $T = \{0, 1\}$ and P is

$$S \rightarrow AA \mid 0$$

$$A \rightarrow SS \mid 1$$

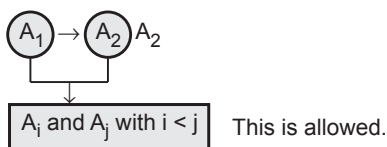
Solution :

Step 1 : The grammar is simplified and it is already in Chomsky's normal form. So we will number out the non-terminal symbols.

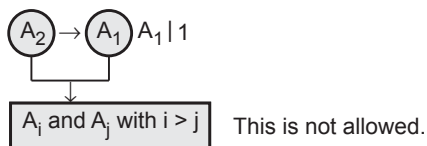
$$S \rightarrow AA \mid 0 \text{ becomes } A_1 \rightarrow A_2 A_2 \mid 0$$

$$A \rightarrow SS \mid 1 \text{ becomes } A_2 \rightarrow A_1 A_1 \mid 1$$

Step 2 : Consider



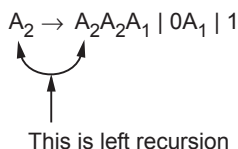
But



Hence we will replace A_1 on RHS by its definition. So we get

$$A_2 \rightarrow A_2 A_2 A_1 \mid 0 A_1 \mid 1$$

Step 3 : Now consider the newly obtained rule



The left recursion can be removed using the following formula

$$A \rightarrow A\alpha \mid \beta \text{ gives } A \rightarrow \beta A' \mid \beta$$

$$A' \rightarrow \alpha A' \mid \alpha$$

Hence,

$$A_2 \rightarrow A_2 A_2 A_1 \mid 0A_1 \mid 1$$

$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ \alpha & \beta_1 & \beta_2 \end{array}$

We get,

$$A_2 \rightarrow 0A_1A' \mid 1A' \mid 0A_1 \mid 1 \quad (1)$$

$$A' \rightarrow A_2A_1A' \mid A_2A_1$$

Thus now if we observe A_2 then it is there in GNF.

Step 4 : Now consider rule for A_1 .

$$A_1 \rightarrow A_2A_2 \mid 0 \quad (2)$$

We will use equation (1) for equation (2) and replace A_2 on R.H.S.

$$\therefore A_1 \rightarrow 0A_1A'A_2 \mid 1A'A_2 \mid 0A_1A_2 \mid 1A_2 \mid 0$$

Step 5 : Now both A_1 and A_2 are in GNF. The only remaining non-terminal which is not in GNE is our newly introduced A' . The A' rule is

$$A' \rightarrow A_2A_1A' \mid A_2A_1 \quad (3)$$

Again use equation (1) and replace A_2 of equation (3)

$$\therefore A' \rightarrow 0A_1A'A_1A' \mid 1A'A_1A' \mid 0A_1A_1A' \mid 1A_1A'$$

$$A' \rightarrow 0A_1A'A_1 \mid 1A'A_1 \mid 0A_1A_1 \mid 1A_1$$

Step 6 : Thus now all the rules are in GNF. The rules can be summarized as follows

$$A_1 \rightarrow 0A_1A'A_2 \mid 1A'A_2 \mid 0A_1A_2 \mid 1A_2 \mid 0$$

$$A_2 \rightarrow 0A_1A' \mid 1A' \mid 0A_1 \mid 1$$

$$A' \rightarrow 0A_1A'A_1A' \mid 1A'A_1A' \mid 0A_1A_1A' \mid 1A_1A'$$

$$A' \rightarrow 0A_1A'A_1 \mid 1A'A_1 \mid 0A_1A_1 \mid 1A_1$$

Example 3.9.14 Convert the grammar $S \rightarrow AB, A \rightarrow BS \mid b, B \rightarrow SA \mid a$ into Greibach normal form.

Solution :

Step 1 : The grammar is simplified and is already in CNF. So we will number out the non terminals as $A_1, A_2, A_3, \dots$

$$S = A_1, \quad A = A_2, \quad B = A_3$$

$$A_1 \rightarrow A_2A_3$$

$$A_2 \rightarrow A_3A_1 \mid b$$

$$A_3 \rightarrow A_1A_2 \mid a$$

Step 2 : For the rules

$$A_1 \rightarrow A_2 A_3$$

$$A_2 \rightarrow A_3 A_1 | b$$

with $A_i \rightarrow A_j A_k \dots$, the $i < j$

But for

$$A_3 \rightarrow A_1 A_2 | a \quad i > j \text{ i.e. } 3 > 1$$

Hence we will process A_3 rule first

$$A_3 \rightarrow A_1 A_2 | a$$

$$A_3 \rightarrow A_2 A_3 A_2 | a \quad \because A_1 \rightarrow A_2 A_3$$

$$A_3 \rightarrow A_3 A_1 A_3 A_2 | b A_3 A_2 | a \quad \because A_2 \rightarrow A_3 A_1 | b$$

Step 3 : Consider newly obtained rule

The left recursion can be eliminated as

$$A_3 \rightarrow A_3 A_1 A_3 A_2 | b A_3 A_2 | a$$


left recursion

$$A \rightarrow A\alpha | \beta \quad \text{then} \quad A \rightarrow \beta A' | \beta$$

$$A' \rightarrow \alpha A' | \alpha$$

Hence

$$\therefore A_3 \rightarrow b A_3 A_2 A' | a A' \quad (1)$$

$$A' \rightarrow A_1 A_3 A_2 A' | A_1 A_3 A_2$$

$$A_3 \rightarrow \underbrace{A_3 A_1 A_3 A_2}_{\alpha} | \underbrace{b A_3 A_2}_{\beta_1} | \underbrace{a}_{\beta_2}$$

Now rule for A_3 is in GNF

Step 4 : Now consider rule for A_2

$$A_2 \rightarrow A_3 A_1 | b \quad (2)$$

Substitute equation 1 in equation 2

$$A_2 \rightarrow b A_3 A_2 A' A_1 | a A' A_1 | b \quad (3)$$

Note that now A_2 is also in GNF

Step 5 : Now we will consider rule for A_1

$$A_1 \rightarrow A_2 A_3 \quad (4)$$

Substitute equation (3) in equation (4),

$$A_1 \rightarrow bA_3 A_2 A' A_1 A_3 | aA' A_1 A_3 | bA_3 \quad (5)$$

Now A_1 is also in GNF.

Step 6 : Now the only remaining non terminal in A'

$$A' \rightarrow A_1 A_3 A_2 A' | A_1 A_3 A_2 \quad (6)$$

Substitute equation (5) in equation (6)

We will get

$$\begin{aligned} A' \rightarrow & bA_3 A_2 A' A_1 A_3 A_3 A_2 A' | aA' A_1 A_3 A_1 A_3 A_2 A' \\ & | bA_3 A_3 A_2 A' | bA_3 A_2 A' A_1 A_3 A_3 A_2 | aA' A_1 A_3 A_3 A_2 | bA_3 A_3 A_2 \end{aligned}$$

Thus A' is now in GNF.

Step 7 : Thus all the rules are in GNF. The rules can be summarized as

$$\begin{aligned} A_1 &\rightarrow bA_3 A_2 A' A_1 A_3 | aA' A_1 A_3 | bA_3 \\ A_2 &\rightarrow bA_3 A_2 A' A_1 | aA' A_1 | b \\ A_3 &\rightarrow bA_3 A_2 A' | aA' \\ A' &\rightarrow bA_3 A_2 A' A_1 A_3 A_3 A_2 A' | aA' A_1 A_3 A_1 A_3 A_2 A' | \\ & bA_3 A_3 A_2 A' | bA_3 A_2 A' A_1 A_3 A_3 A_2 | aA' A_1 A_3 A_3 A_2 | bA_3 A_3 A_2 \end{aligned}$$

Review Question

1. What is a normal form ? Explain Chomsky Normal Form (CNF) and Greibach Normal Form (GNF) with suitable example.

SPPU : May-08, Marks 8, Dec.-17, 19, End Sem, May-18, End Sem, Marks 8

3.10 Pumping Lemma for CFG

- The pumping lemma for regular sets states that every sufficiently long string in a regular set contains a short substring that can be pumped. In other words, if a long string is given and if we **push or pump** any number of substrings in any number of times then we always get a regular set.
- According to pumping lemma for CFL's there are always two short substrings which are close to each other and these both the substrings can be repeated as many times as required.

Lemma : Let L be any context free language, then there is a constant n , which depends only upon L , such that there exist a string $w \in L$ and $|w| \geq n$ where $w = pqrst$ such that

1. $|qsl| \geq 1$
2. $|prst| \leq n$ and
3. For all $i \geq 0$ $p q^i r s^i t$ is in L .

Proof :

- This pumping lemma states that if there is a language L which is without unit productions and without a null production and there exist w where $w \in L$.
- The string w can be derived by a context free grammar G . The G be a grammar which is in Chomsky's Normal Form.
- The grammar G generates language L . For the string w , we can obtain a parse tree which derives the string w . Then if the length of the path to w is less than equal to i then the length of the word w is less than or equal to 2^{i-1} .
- We can prove this by induction step.

Basis : If $i = 1$

Let G contains the rule $S \rightarrow a$ where length of the derived string is 1 i.e. $i = 1$. Now according to the rule the word length should be $\leq 2^{i-1}$ i.e. $2^0 = 1$. Observe that we have a word ' a ' which is of length 1. Also observe that the grammar G is in Chomsky's Normal Form. This language is regular since $|w| = |pqrst| = 1$.

S
|
a

Fig. 3.10.1

Induction step : Let w be a string which is derived by grammar G . Let k be a variable such that $n = 2^k$, $|w| \geq n$ then $|w| > 2^{k-1}$ while deriving w string we may get some non-terminals of CFG - G can be repeated for any number of times and will give the string w . If we pump the substrings to w such that the path length of this newly formed string w' (w + pumped string = w') is i and the word length of w' is $\leq 2^{i-1}$ then the grammar G deriving w' is called a regular grammar. The necessary condition is that grammar G is in Chomsky's Normal Form.

Let us consider a grammar,

$$G = (\{A, B, C\}, \{a\}, \{A \rightarrow BC$$

$$B \rightarrow BA$$

$$C \rightarrow BA$$

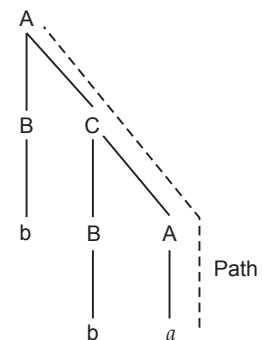


Fig. 3.10.2

$$A \rightarrow a$$

$$B \rightarrow b\}, A)$$

Thus

$$A \xRightarrow{*} bba = w$$

i.e. path length $i = 3$

$$|w| \leq 2^{i-1} \text{ i.e. } 3 \leq 2^2$$

If we pump a substring into w which satisfies the condition as $i \leq |w| \leq 2^{i-1} \leq n$ the grammar producing string w is a regular grammar.

Example 3.10.1 Using the CFL pumping lemma, show that the following language is not context free. $L = \{a^i b^j c^k \mid i < j < k\}$.

Solution : Let us prove this statement by method of contradiction. Assume L is a context free language. And w be any string such that $w \in L$.

Let, w be

$$w = p q r s t$$

Let, $|qsl| \geq 1$ and $|p r s| \leq n$ and the $p r s$ can contain at the most strings of a 's and c 's. The string $p q^i r s^i t$ contains additional occurrences of q 's and s 's.

As per the condition mentioned in a lemma, $|qsl| \geq 1$ this means that either q or s contains atleast one a then qs can not contain any c 's and as per the language L the number of c 's has to be more than number of a 's. Therefore the $w \notin L$. Similarly if $w = p q^0 r s^0 t$ then also there are fewer occurrences of c 's than a 's which can not be our L . Hence the assumption which we had made that L is a context free language is wrong.

Example 3.10.2 Show that $L = \{a^n b^n c^n \mid n \geq 0\}$ is not a context free language.

Solution : Let us assume that

$$L = a^n b^n c^n \text{ is a context free language.}$$

Let, w be any string such that $w \in L$.

Let,

$$w = pqrst$$

Let, $|qsl| \geq 1$ $|prsl| \leq n$

Now let us consider

$$w = pq^i rs^i t$$

i.e. we have additional occurrences of q and s . Consider various cases.

Case 1 : Consider $i = 0$ then,

$$w = pq^i rs^i t$$

if $i = 0$ then q^0 and s^0 . That means q and s are absent then,

$$w = prt$$

$$w = \begin{array}{cccc} aa & bb & cc \\ \square & \square & \square & \square \\ p & q & r & s & t \end{array}$$

$$= a^2bc$$

$$\text{i.e.} \quad = aabc \notin L$$

Hence our assumption of L being CFG is wrong.

Case 2 : Consider $i = 2$ then,

$$w = pq^i rs^i t$$

$$= pqqrsst$$

Consider,

$$w = \begin{array}{cccc} aa & bb & cc \\ \square & \square & \square & \square \\ p & q & r & s & t \end{array}$$

Now with $i = 2$ we get,

$$= a^2 b^3 c^3 \notin L$$

$$w = \begin{array}{cccc} aa & bbb & ccc \\ \square & \square & \square & \square \\ p & qq & r & ss & t \end{array}$$

Thus our assumption of L being CFG is wrong. This proves that the given language L is not a context free.

3.11 Closure Properties of CFL

SPPU : May 09, Dec.-18, Marks 6

The regular languages are closed under union, concatenation and kleen closure. Now we will discuss closure properties of context free languages. In the sense, we will, check which are those properties for them CFLs are closed under. The context free languages are closed under some operation means after performing that particular operation on those CFLs the resultant language is context free language. These properties are as below.

1. The context free languages are closed under union.
2. The context free languages are closed under concatenation.
3. The context free languages are closed under kleen closure.
4. The context free languages are not closed under intersection.
5. The context free languages are not closed under complement.

We will discuss the above mentioned closure properties of CFL with the help of proofs and examples.

Theorem 1 : If L_1 and L_2 are context free languages then
 $L = L_1 \cup L_2$ is also context free. That is, the CFLs are closed under union.

Proof : We will consider two languages L_1 and L_2 which are context free languages. We can give these languages using context free grammars G_1 and G_2 such that $G_1 \in L_1$ and $G_2 \in L_2$. The G_1 can be given as $G_1 = \{V_1, \Sigma, P_1, S_1\}$ where P_1 can be given as

$$P_1 = \{$$

$$S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid \epsilon$$

$$A_1 \rightarrow a$$

$$B_1 \rightarrow b$$

$$\}$$

Here $V_1 = \{S_1, A_1, B_1\}$ and S_1 is a start symbol.

Similarly, we can write $G_2 = \{V_2, \Sigma, P_2, S_2\}$

$N_2 = \{S_2, A_2, B_2\}$ and S_2 is a start symbol.

P_2 can be given as :

$$P_2 = \{$$

$$S_2 \rightarrow a A_2 A_2 \mid b B_2 B_2$$

$$A_2 \rightarrow b$$

$$B_2 \rightarrow a$$

$$\}$$

Now $L = L_1 \cup L_2$ gives $G \in L$. This G can be written as

$$G = \{V, \Sigma, P, S\}$$

$$V = \{S_1, A_1, B_1, S_2, A_2, B_2\}$$

$$P = \{P_1 \cup P_2\}$$

S is a start symbol

$$P = \{ S \rightarrow S_1 \mid S_2$$

$$S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid \epsilon$$

$$A_1 \rightarrow a$$

$$B_1 \rightarrow b$$

$$S_2 \rightarrow aA_2A_2 \mid bB_2B_2$$

$$A_2 \rightarrow b$$

$$B_2 \rightarrow a$$

}

Thus grammar G is a context free grammar which produces languages L which is context free language.

Theorem 2 : If L_1 and L_2 are two context free languages then L_1L_2 is CFG. That means context free languages are closed under concatenation.

Proof : Let L_1 is a context free language which can be represented by a context free grammar G_1 , such that $G_1 \in L_1$ and

$$G_1 = \{V_1, \Sigma, P_1, S_1\}$$

$$V_1 = \{S_1, A_1, B_1\}$$

$$\Sigma = \{a, b\}$$

S_1 is a start symbol and P_1 is a set of production rules,

$$P_1 = \{ S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid \epsilon$$

$$A_1 \rightarrow a$$

$$B_1 \rightarrow b$$

}

Similarly, L_2 is a context free language which can be represented by a context free grammar G_2 , such that $G_2 \in L_2$ and

$$G_2 = \{V_2, \Sigma, P_2, S_2\}$$

$$V_2 = \{S_2, A_2, B_2\}$$

$$\Sigma = \{a, b\}$$

S_2 is a start symbol and P_2 is a set of production rules,

$$P_2 = \{ S_2 \rightarrow aA_2A_2 \mid bB_2B_2$$

$$A_2 \rightarrow b$$

$$B_2 \rightarrow a$$

}

Now $L = L_1 L_2$ can be obtained by G such that $G = G_1 \cup G_2$. Therefore

$$G = \{V, \Sigma, P, S\}$$

$$V = \{S, S_1, A_1, B_1, S_2, A_2, B_2\}$$

where S is a start symbol. The production rules, P can be given as,

$$\begin{aligned}
 P = \{ & S \rightarrow S_1 \mid S_2 \\
 & S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid \epsilon \\
 & A_1 \rightarrow a \\
 & B_1 \rightarrow b \\
 & S_2 \rightarrow a A_2 A_2 \mid b B_2 B_2 \\
 & A_2 \rightarrow b \\
 & B_2 \rightarrow a \\
 & \}
 \end{aligned}$$

As grammar G is context free grammar the language L produced by G is also context free language. Hence context free languages are closed under concatenation.

Theorem 3 : If L_1 is context free language then L_1^* is also context free. That means CFL is closed under kleen closure.

Proof : Let, L_1 be a context free language represented by G_1 such that $G_1 \rightarrow \epsilon L_1$.

The CFG G_1 can be given as

$G_1 = \{V_1, \Sigma, P_1, S_1\}$ where S_1 is a start symbol

$$\begin{aligned}
 P_1 = \{ & S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1 \mid \epsilon \\
 & A_1 \rightarrow a \\
 & B_1 \rightarrow b \\
 & \}
 \end{aligned}$$

Now $L = L_1^*$ can be represented by a grammar G such that

$$G = \{ (V, \Sigma, P, S) \}$$

$$V = \{S, S_1, A_1, B_1\}$$

and $P = S \rightarrow S_1 S \mid \epsilon$

$$S_1 \rightarrow A_1 S_1 A_1 \mid B_1 S_1 B_1$$

$$A_1 \rightarrow a$$

$$B_1 \rightarrow b$$

}

Thus grammar G is a context free grammar and language L produced by G is also context free language. Hence context free language are closed under kleen closure.

Theorem 4 : If L_1 and L_2 are two CFLs then $L = L_1 \cap L_2$ may be CFL or may not be CFL. That means L is not closed under intersection.

Proof : Let, $L_1 = \{0^n 1^n 2^i \mid n \geq 1, i \geq 1\}$

$$L_2 = \{0^n 1^n 2^n \mid n \geq 1\}$$

The grammar for L_1 is

$$S \rightarrow AB$$

$$A \rightarrow 0A1 \mid 01$$

$$B \rightarrow 2B \mid 2$$

Similarly L_2 can be represented by grammar.

$$S \rightarrow AB$$

$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B2 \mid 12$$

Now if we try to obtain

$L = L_1 \cap L_2$ then we get sometimes context free languages and sometimes non context free languages. Thus we can say that CFLs are not closed under intersection.

Theorem 5 : If L_1 is a CFL then L'_1 may or may not be CFL. That means CFL is not closed under complement.

Proof : Let L_1 and L_2 are two CFLs. We will assume that complement of a context free language is a CFL itself. Hence L'_1 and L'_2 both are CFLs. We can also state that $(L'_1 \cup L'_2)$ is context free (since CFLs are closed under union). But $(L'_1 \cup L'_2) = L_1 \cap L_2$ i.e. $L = L_1 \cap L_2$ may or may not be CFL. The L_1 and L_2 are arbitrary CFLs, there may exist L'_1 and L'_2 which are not CFL. Hence complement of certain language may be context free or may not be. Therefore we can say that CFL is not closed under complement operation.

Theorem 6 : Context Free Languages are closed under homomorphism.

Proof : Let, $G = (V, T, P, S)$ be the context free grammar for generating the language L . The input set is Σ and let Γ be the set of substitution symbols. The $\Sigma^* \rightarrow \Gamma^*$ be a homomorphism.

Let $a \in L$ and w be the substitution that replaces every $a \in \Sigma$.

Then $h(w) = h(a_1)h(a_2)h(a_3)$ which ultimately generates $h(L)$. Thus $h(a_i)$ is homomorphism for every $a_i \in \Sigma$.

For example : Consider

$$P = S \rightarrow aSb \mid bSb \mid \epsilon$$

Let $h(a) = 00$

$$h(b) = 11$$

Then $h(w) = S \rightarrow 00S11 \mid 11S00 \mid \epsilon$

is again a GFG.

Hence it is proved that context free languages are closed under homomorphism.

Review Questions

1. Write a note on closure properties of CFL.

SPPU : May 09, Marks 4

2. Show that CFLs are not closed under intersection and complementation.

SPPU : Dec.-18, End Sem, Marks 6

3. Prove that CFLs are closed under union, concatenation and Kleen's closure.

SPPU : Dec.-18, End Sem, Marks 6

3.12 Decision Properties of CFL

SPPU : Dec.-11, Marks 5

Various decision properties of CFLs are

1. Emptiness
2. Finiteness
3. Membership

These properties are discussed in detail as follows.

1. Emptiness

Theorem : There exists an algorithm which can determine whether or not the given context free grammar can generate any word at all.

Proof : The algorithm to determine whether or not a grammar generates any word is given below -

- i) Put the dot ($\bullet$) above every terminal symbol. If there is any null string ϵ then put dot over ϵ as well. These symbols occur at right hand side of production rule.
- ii) If all the symbols of right hand side of the production rule are dotted then dot the corresponding non terminal (which is at left hand side of production rule) throughout the grammar. Repeat this step as long as possible.
- iii) If the starting symbol gets dotted then answer is yes otherwise answer is no.

Let us apply this algorithm on some grammar.

Let,

$$S \rightarrow PQ$$

$$P \rightarrow AP \mid AA$$

$$A \rightarrow a$$

$$Q \rightarrow BQ \mid BB$$

$$B \rightarrow b$$

Now if we want to put dots we will put over terminals at RHS of rule.

Step 1 :

$$1) \quad S \rightarrow PQ$$

$$2) \quad P \rightarrow AP \mid AA$$

$$3) \quad A \rightarrow \dot{a}$$

$$4) \quad Q \rightarrow BQ \mid BB$$

$$5) \quad B \rightarrow \dot{b}$$

Step 2 :

As we RHS of rule 3) and RHS of rule 5) is dotted we will dot A and B throughout the grammar.

$$1) \quad S \rightarrow PQ$$

$$2) \quad P \rightarrow \dot{A}P \mid \dot{A}\dot{A}$$

$$3) \quad \dot{A} \rightarrow \dot{a}$$

$$4) \quad Q \rightarrow \dot{B}Q \mid \dot{B}\dot{B}$$

$$5) \quad \dot{B} \rightarrow \dot{b}$$

Now, in rule 2) the alternate production at RHS is dotted $\left(\dot{A}\dot{A}\right)$. Hence dot P.

Similarly dot Q.

Hence,

$$1) \quad S \rightarrow \dot{P}\dot{Q}$$

$$2) \quad \dot{P} \rightarrow \dot{A}\dot{P} \mid \dot{A}\dot{A}$$

$$3) \quad \dot{A} \rightarrow \dot{a}$$

$$4) \quad \dot{Q} \rightarrow \dot{B}\dot{Q} \mid \dot{B}\dot{B}$$

$$5) \quad \dot{B} \rightarrow \dot{b}$$

Thus in rule 1) the R.H.S. is dotted. Hence dot S. But since S being dotted is a start symbol, the algorithm generates answer yes. Hence we can prove that an algorithm exists which determines whether or not the given grammar generates any string.

Finiteness

Theorem : There exists an algorithm to determine whether the given context free grammar generates a finite or infinite language.

SPPU : Dec.-11, Marks 5

Proof : We will write following algorithm -

1. Make the grammar simplified by removing unit productions and useless symbols.
2. Check whether there is any self-embedded non terminal or not by following steps.
 - a) Underline some non terminal say X which is at LHS of production rule.
 - b) Then put dot over all the X which are at RHS throughout the grammar. The dotted X and underlined X are treated as two different symbols.
 - c) Put dot over any non terminal at LHS whose RHS contains any dotted symbols. Dot this non terminal throughout the grammar.
 - d) If underlined X gets dotted then X is called self embedded otherwise it is not self embedded.
3. If the grammar contains any self embedded non terminal then it generates infinite languages otherwise it generates finite languages.

To support this algorithm, let us take one grammar.

$$S \rightarrow PQa \mid bpz \mid b$$

$$P \rightarrow Xb \mid bZa$$

$$Q \rightarrow bPP$$

$$X \rightarrow aZa \mid aaa$$

$$Z \rightarrow ZPbP$$

The above grammar has Z as useless symbol. Hence by removing it we get,

$$1) S \rightarrow PQa \mid b$$

$$2) P \rightarrow Xb$$

$$3) Q \rightarrow bPP$$

$$4) X \rightarrow aaa$$

Now we will mark X of rule 4) underlined and put dot over x in rule 2)

$$1) \quad S \rightarrow PQa \mid b$$

$$2) \quad P \rightarrow \dot{X}b$$

$$3) \quad Q \rightarrow bPP$$

$$4) \quad \underline{X} \rightarrow aaa$$

As R.H.S. of rule 2) contains one dotted symbol dot P throughout the grammar

$$S \rightarrow \dot{P}Qa \mid b$$

$$\dot{P} \rightarrow \dot{X}b$$

$$Q \rightarrow b\dot{P}\dot{P}$$

$$\underline{X} \rightarrow aaa$$

Then Q gets dotted

$$S \rightarrow \dot{P}\dot{Q}a \mid b$$

$$\dot{P} \rightarrow \dot{X}b$$

$$\dot{Q} \rightarrow b\dot{P}\dot{P}$$

$$\underline{X} \rightarrow aaa$$

The S gets dotted

$$\dot{S} \rightarrow \dot{P}\dot{Q}a \mid b$$

$$\dot{P} \rightarrow \dot{X}b$$

$$\dot{Q} \rightarrow b\dot{P}\dot{P}$$

$$\underline{X} \rightarrow aaa$$

Now all the non terminals are dotted but X is not. That means X is not self embedded symbol. Hence the grammar generates finite language.

Membership : There exists an algorithm which tells whether given string belongs to given grammar.

To check whether the given grammar generates desired string the derivation tree can be drawn.

Consider the grammar,

$$S \rightarrow PQa \mid b$$

$$P \rightarrow Xb$$

$$Q \rightarrow bPP$$

$$X \rightarrow aaa$$

Check whether it generates string aaabbbaaabaaba.

The leaves generated string aaabbbaaabaaba.

3.13 Chomsky Hierarchy

SPPU : May-06, Dec.-11, Marks 8

The Chomsky's Hierarchy represents the class of languages that are accepted by different machine. The category of languages in Chomsky's Hierarchy is as given below -

Language class	Language	Grammar	Machine	One example
Type 3	Regular	Regular grammar	FSM i.e. NFA or DFA	a^*b^*
Type 2	Context free	Context free grammar	PDA	$a^n b^n$
Type 1	Decidable languages	Context sensitive grammar	Linear bounded automata	$a^n b^n c^n$
Type 0	Computable languages	Unrestricted grammar	Turing machine	$n!$

This is a hierarchy therefore every language of type 3 is also of type 2, 1 and 0. Similarly every language of type 2 is also of type 1 and 0 etc.

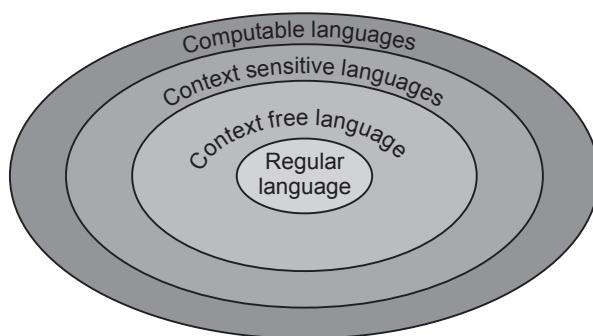


Fig. 3.13.1 Chomsky hierarchy

Type 3 - Regular languages

Regular languages are those languages which can be described using regular expressions. These languages can be modelled by NFA or DFA.

Type 2 - Context free languages

The context free languages are the languages which can be represented by Context Free Grammar (CFG). The production rule is of the form

$$A \rightarrow \alpha$$

where A is any single non-terminal and α is any combination of terminals and non-terminals.

A NFA or DFA cannot recognize strings of this language because these automata are not having "stack" to memorize. Instead of it the Push Down Automata can be used to represent these languages.

Type 1 - Context sensitive languages

The context sensitive grammars are used to represent context sensitive languages. The context sensitive grammar follows the following rules -

1. The context sensitive grammar may have more than one symbol on the left hand side of their production rules.
2. The number of symbols on the left hand side must not exceed the number of symbols on the right hand side.
3. The rule of the form $A \rightarrow \epsilon$ is not allowed unless A is a start symbol. It does not occur on the right hand side of any rule.

The automaton which recognizes context sensitive languages is called linear bounded automaton. While deriving using context sensitive grammar the sentential form must always increase in length every time a production rule is applied. Thus the size of a sentential form is bounded by a length of the sentence we are deriving.

Type 0 - Unrestricted languages

There is no restriction on the grammar rules of these type of languages. These languages can be effectively modeled by Turing machines.

Review Questions

1. Define Type 0 and Type 1 grammar.
2. Write short note on Chomsky hierarchy.

SPPU : May-06, Marks 4

SPPU : Dec.-11, Marks 8

3.14 Cock-Younger-Kasami Algorithm

- Cock-Younger-Kasami or CYK algorithm is used to decide whether a string belongs to language of grammar or not.

- It is also called as **membership algorithm** of context free grammar.
- CYK algorithm works only on CFG which are in Chomsky-Normal-Form (CNF).

Algorithm

```

Begin
for ( i = 1 to n do )
Vi1 { A | A → a is a production where ith symbol of x is a }
for ( j = 2 to n do )
for ( i = 1 to n - j + 1 do )
Begin
Vij = → Φ
For k = 1 to j - 1 do
Vij = Vij → ∪
{A | A → BC is a production where B is in Vik and C is in V(i + k)(j - k) }
End
End

```

Let us understand this algorithm with some suitable example.

Consider the following grammar to check the acceptance of the string **w = baaba**

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

The string can be represented in a triangular table as -

The string 'x' has following positions -

x ₁₅				
x ₁₄	x ₂₄			
x ₁₃	x ₂₃	x ₃₃		
x ₁₂	x ₂₂	x ₃₂	x ₄₂	
x ₁₁	x ₂₁	x ₃₁	x ₄₁	x ₅₁

baaba				
baab	aaba			
baa	aab	aba		
ba	aa	ab	ba	
b	a	a	b	a

Notation for Non-terminal V_{ij}

The V_{ij} represents a **set of variables** in the grammar which can drive the substring x_{ij} .

If set of variables contain start symbol then -

- 1) The substring x_{ij} can be derived from given grammar.
- 2) The substring x_{ij} is a member of the language of the given grammar.

The triangular table for V_{ij} is as given below -

V_{15}				
V_{14}	V_{24}			
V_{13}	V_{23}	V_{33}		
V_{12}	V_{22}	V_{32}	V_{42}	
V_{11}	V_{21}	V_{31}	V_{41}	V_{51}

Step 1 : We will fill up x_{ij} and V_{ij} table as -

- **Finding x_{11} and V_{11}** - The V_{11} represents the set of variables deriving x_{11} . Here $x_{11} = b$. From given CFG, only non-terminal B derives 'b' in given grammar.

$$\therefore x_{11} = b, \quad V_{11} = \{B\}$$

- **Finding x_{21} and V_{21}** - The V_{21} represents the set of variables deriving x_{21} . Here $x_{21} = a$. From given CFG, only non-terminals A, C derives 'a' in given grammar.

$$\therefore x_{21} = a, \quad V_{21} = \{A, C\}$$

- **Finding x_{31} and V_{31}** - The V_{31} represents the set of variables deriving x_{31} . Here $x_{31} = a$. From given CFG, only non-terminals A and C derive 'a'.

$$\therefore x_{31} = a, \quad V_{31} = \{A, C\}$$

- **Finding x_{41} and V_{41}** - The V_{41} represents the set of variables deriving x_{41} . Here $x_{41} = b$. From given CFG, the non-terminal B derives 'b'.

$$\therefore x_{41} = b, \quad V_{41} = \{B\}$$

- **Finding x_{51} and V_{51}** - The V_{51} represents the set of variables deriving x_{51} . Here $x_{51} = a$. From given CFG, the non-terminals A and C derive 'a'.

$$\therefore x_{51} = a, \quad V_{51} = \{A, C\}$$

The x_{ij} and V_{ij} table, partially filled up with bottom-most row is -

The x_{ij} table

b	a	a	b	a

The V_{ij} table

{B}	{A, C}	{A, C}	{B}	{A, C}

Step 2 : Now for filling up the table in upward direction from the 2nd row from bottom, we will use following formula -

$$V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

where k varies from 1 to j - 1.

- **Finding x_{12} and V_{12}** - We have i = 1, j = 2, k = 1

Here $x_{12} = \text{'ba'}$

$$\begin{aligned} V_{12} &= V_{11} \cup V_{(1+1)(2-1)} \\ &= V_{11} \cup V_{21} \\ &= \{B\} \cup \{A, C\} \\ &= \{BA, BC\} \end{aligned}$$

$$\therefore V_{12} = \{A, S\}$$

- **Finding x_{22} and V_{22}** - We have i = 2, j = 2, k = 1

Here $x_{22} = \text{'aa'}$

$$\begin{aligned} V_{ij} &= V_{ik} \cup V_{(i+k)(j-k)} \\ V_{22} &= V_{21} \cup V_{31} \\ V_{22} &= \{A, C\} \cup \{A, C\} \\ &= \{AA, AC, CA, CC\} \end{aligned}$$

The AA, AC and CA do not exist.

$$\therefore V_{22} = \{CC\}$$

$$\therefore V_{22} = \{B\}$$

- **Finding x_{32} and V_{32}** - We have i = 3, j = 2, k = 1

Here $x_{32} = \text{'ab'}$

$$\begin{aligned} V_{ij} &= V_{ik} \cup V_{(i+k)(j-k)} \\ V_{32} &= V_{31} \cup V_{41} \\ V_{32} &= \{A, C\} \cup \{B\} \\ &= \{AB, CB\} \end{aligned}$$

As 'CB' can not be derived from any non-terminal,

$$V_{32} = \{AB\}$$

$$\therefore V_{32} = \{S, C\}$$

- **Finding x_{42} and V_{42}** - We have $i = 4, j = 2, k = 1$

Here $x_{42} = \text{'ba'}$

$$V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

$$V_{42} = V_{41} \cup V_{51}$$

$$= \{B\} \cup \{A, C\}$$

$$= \{BA, BC\}$$

$$\therefore V_{42} = \{A, S\}$$

The partially filled x_{ij} and V_{ij} tables are

The x_{ij} table

ba	aa	ab	ba	
b	a	a	b	a

The V_{ij} table

{A, S}	{B}	{S, C}	{A, S}	
{B}	{A, C}	{A, C}	{B}	{A, C}

Step 3 :

- **Finding x_{13} and V_{13}** - Here $i = 1, j = 3, k = j - 1, (1, 2)$

$$V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

$$V_{13} = V_{11} \cup V_{22} \cup V_{12} \cup V_{41}$$

$$= \{B\} \cup \{B\} \cup \{A, S\} \cup \{A, C\}$$

$$= \{BB\} \cup \{AA, SA, AC, SC\}$$

As we can not get BB, AA, SA, AC, SC.

$$\therefore V_{13} = \phi \cup \phi$$

$$V_{13} = \emptyset$$

- **Finding x_{23} and V_{23}** - Here $i = 2, j = 3, k = (3 - 1) = (1, 2)$

$$x_{23} = aab$$

$$V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

$$V_{23} = V_{21} \cup V_{32} \cup V_{22} \cup V_{41}$$

$$= \{A, C\} \cup \{S, C\} \cup \{B\} \cup \{B\}$$

$$= \{AS, AC, CS, CC\} \cup \{BB\}$$

Out of the above, only CC can be obtained.

$$\therefore V_{23} = \{CC\}$$

$$\therefore V_{23} = \{B\}$$

• **Finding x_{33} and V_{33} -**

$$x_{33} = aba$$

$$V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

We have $i = 3, j = 3, k = 1$ to $(3 - 1) = 1, 2$

Substituting values in the formula, we get -

$$V_{33} = V_{31} \cup V_{42} \cup V_{32} \cup V_{51}$$

$$V_{33} = \{A, C\} \cup \{A, S\} \cup \{S, C\} \cup \{A, C\}$$

$$V_{33} = \{AA, AS, CA, CS\} \cup \{SA, SC, CA, CC\}$$

Since AA, AS, CA, CS, SA, SC and CA do not exist, so we have -

$$V_{33} = \phi \cup \{CC\}$$

$$V_{33} = \phi \cup \{B\}$$

$$\therefore V_{33} = \{B\}$$

The partially filled table is -

The x_{ij} table

baa	aab	aba		
ba	aa	ab	ba	
b	a	a	b	a

The V_{ij} table

ϕ	$\{B\}$	$\{B\}$		
$\{A, S\}$	$\{B\}$	$\{S, C\}$	$\{A, S\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$

Step 4 :

• **Finding x_{14} and V_{14} -**

$$x_{14} = baab$$

Here $i = 1, j = 4, k = j - 1 = 4 - 1 = 1, 2, 3$

$$\therefore V_{ij} = V_{ik} \cup V_{(i+k)(j-k)}$$

$$V_{14} = V_{11} \cup V_{23} \cup V_{12} \cup V_{32} \cup V_{13} \cup V_{41}$$

$$\begin{aligned}
&= \{B\} \{B\} \cup \{A, S\} \{S, C\} \cup \{\phi, B\} \\
&= \{BB, AS, AC, SS, SC, B\}
\end{aligned}$$

As none of the above production exist,

$$\therefore V_{14} = \phi$$

• **Finding x_{24} and V_{24} -**

$$x_{24} = aaba$$

Here $i = 2, j = 4, k = j - 1 = 4 - 1 = 1, 2, 3$

$$\begin{aligned}
V_{ij} &= V_{ik} \cup V_{(i+k)(j-k)} \\
V_{24} &= V_{21} \cup V_{33} \cup V_{22} \cup V_{42} \cup V_{23} \cup V_{51} \\
&= \{A, C\} \{B\} \cup \{B\} \{A, S\} \cup \{B\} \{A, C\} \\
&= \{AB, CB\} \cup \{BA, BS\} \cup \{BA, BC\}
\end{aligned}$$

The CB can not be obtained

$$= \{AB, BA, BS, BC\}$$

$$\therefore V_{24} = \{S, C, A\}$$

Step 5 : Finding x_{15} and V_{15} -

$$x_{15} = baaba$$

Here $i = 1, j = 5, k = 5 - 1 = 1, 2, 3, 4$

$$\begin{aligned}
V_{ij} &= V_{ik} \cup V_{(i+k)(j-k)} \\
V_{15} &= V_{11} \cup V_{24} \cup V_{12} \cup V_{33} \cup V_{13} \cup V_{42} \cup V_{14} \cup V_{51} \\
V_{15} &= \{B\} \{S, C, A\} \cup \{A, S\} \{B\} \cup \{\phi\} \{A, S\} \cup \{\phi\} \{A, C\} \\
V_{15} &= \{BS, BC, BA\} \cup \{AB, SB\} \cup \{A, S\} \cup \{A, C\}
\end{aligned}$$

Since BS, SB, A, S and C do not exist, so we have -

$$V_{15} = \{BC, BA\} \cup \{AB\} \cup \phi \cup \phi$$

$$V_{15} = \{S, A\} \cup \{S, C\} \cup \phi \cup \phi$$

$$\therefore V_{15} = \{S, A, C\}$$

The table is

The x_{ij} table

baaba				
baab	aaba			
baa	aab	aba		
ba	aa	ab	ba	
b	a	a	b	a

The V_{ij} table

{S, A, C}				
ϕ	{S, C, A}			
ϕ	{B}	{B}		
{A, S}	{B}	{S, C}	{A, S}	
{B}	{A, C}	{A, C}	{B}	{A, C}

As we can see, $x_{15} = \text{baaba}$ is a string and $V_{15} = \{S, A, C\}$

That means V_{15} contains start symbol S. Thus $x_{15} = \text{baaba}$ is a member of given grammar. In other words, the given string baaba can be obtained by given grammar.

The value ϕ in V_{ij} table indicates that corresponding string in x_{ij} are not the members of given grammar.

Thus 'baab' and 'baa' can not be obtained from given grammar.

3.15 Applications of CFG SPPU : May-06, 11, Dec.-06, 13, Oct.-16, 19, Marks 6

When any high level program like C or Pascal is compiled, the compiler checks the syntax of every programming statement by constructing syntax tree. And for building the syntax tree, it is necessary to write context free grammar for each statement in the program.

For example, if in your C program the statement is

$$x = y + z;$$

Then the x, y and z are identified as 'id' by lexical analyzer. It will be interpreted as

$$\text{'id'} = \text{'id'} + \text{'id'} ;$$

The CFG for this will be

$$S \rightarrow \text{id} = \text{ET}$$

$$E \rightarrow E + F$$

$$E \rightarrow F$$

$$F \rightarrow \text{id}$$

$$T \rightarrow ;$$

Let us build the parse tree accordingly.

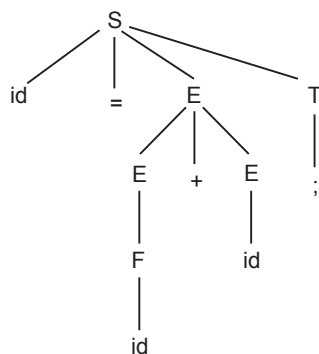


Fig. 3.15.1 Parse tree

Read out the leaf nodes and you will get `id = id + id ;`. If `;` is missing in your statement rule won't get matched with `T` branch and hence it will generate syntax error as 'statement `;` missing'.

Thus context free grammar is a powerful tool used in compilers for syntax analysis.

1. Parsing techniques

Context free grammar is used for parsing the programming constructs and finding the syntactical errors from source program. There are two types of parser that are used by compiler top down parser and bottom up parser.

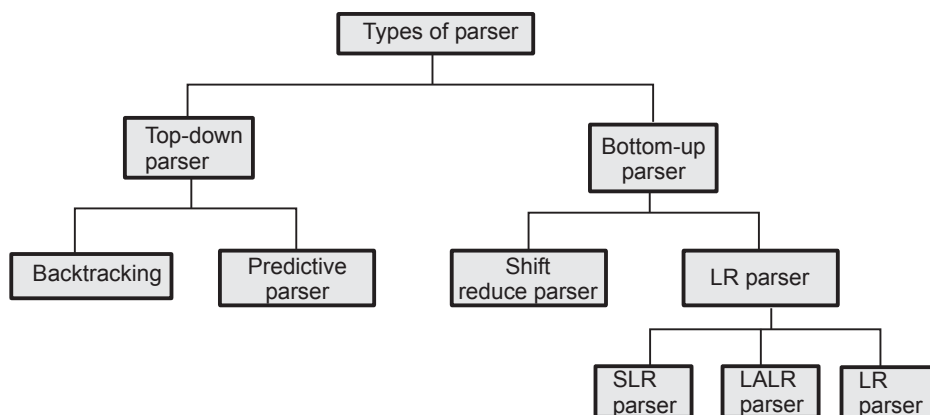


Fig. 3.15.2 Parsing techniques

When parse tree is constructed from root and expanded to leaves then such type of parser is called **Top-down parser**.

When parse tree is constructed from leaves to root, then such type of parser is called **Bottom-up parser**.

2. Markup languages

Markup languages is a family of languages in which the certain strings have special meaning. These strings are referred as **tags**. Thus tags tell us about the semantics of various strings within the document. The most commonly used example of markup language is **Hyper Text Markup Language (HTML)**. The context free language is used to describe the structure of HTML document. Consider following HTML document -

```
<p>Wonderful colors </p>
<h1>Red</h1>
<h2>Blue</h2>
<h3>Green</h3>
```

The above scripting document will help in displaying the text on the web page with special formatting. In order to check this HTML document syntactically, a context free grammar is used. Following are some variables/non terminals are used to parse this HTML document -

1. Text

It represents the collection of strings.

2. Char

It represents a single character which is allowed in the HTML document. The collection of various characters lead to a text.

3. Doc

It represents the sequence of elements.

4. Element

The element can be simple text or a text enclosed within the tags.

The production rules defining above HTML document will be -

$$Char \rightarrow a \mid A \mid \dots$$

$$Text \rightarrow Char \ Text \mid \epsilon$$

$$Doc \rightarrow Element \ Doc \mid \epsilon$$

$$Element \rightarrow Text \mid$$

$$\langle h1 \rangle Doc \langle /h1 \rangle \mid$$

$$\langle h2 \rangle Doc \langle /h2 \rangle \mid$$

$$\langle h3 \rangle Doc \langle /h3 \rangle \mid \dots$$

Thus CFG is used to describe the structure of HTML document.

3. XML and Document Type Definitions

The HTML is a markup language which makes use of some standard tags. There are some special meanings associated with these tags. But Extensible Markup Language (XML) is a special kind of language using which user can define his own tags. The purpose of XML is not to describe the formatting of HTML but to describe the semantics of the text. For example, consider following XML document.

```
<Person>
  <Personal-Info>
    <Name>My Name is Jaya</Name>
    <City>I live in Pune</City>
  </Personal-Info>
  <Hobby>
    <first>I like reading</first>
    <second>I like programming</second>
    <third>I like singing</third>
  </Hobby>
</Person>
```

Note that the XML document allows user to use user-defined tags which are useful for describing the semantics of the text. For instance : if we want to know the city in which the person lives then it becomes convenient to find out the corresponding information. The **Document Type Definition (DTD)** is a kind of context free grammar used to define the structure of the XML document. The DTD has its own notations and variables.

Using DTD we can specify the various elements types, attributes and their relationship with one another. Following example shows how DTD is useful for describing the structure of XML document -

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
  <!ELEMENT student (name,address,std,marks)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT address (#PCDATA)>
  <!ELEMENT std (#PCDATA)>
  <!ELEMENT marks (#PCDATA)>
]>
<student>
```

Here we are defining the
DTD

```

<name>Anand</name>
<address>Pune</address>
<std>Second</std>
<marks>70 percent</marks>
</student>

```

In DTD, The basic entity is **ELEMENT**. The elements are used for defining the tags. The elements typically consist of opening and closing tag. Mostly only one element is used to define a single tag. The PCDATA is used in above given XML document. It stands for **P**arsed **C**haracter **D**ata (i.e. text). Any Parsable character data should not contain the markup characters. The markup characters are < or > or &. If we want to use less than, greater than or ampersand characters then make use of < , > or & The notations of context free grammar can also be used in DTD. For instance :

hobby → programming|reading|singing
can be denoted as :

```
<!ELEMENT hobby(programming | reading |singing )
```

In order to denote more than one occurrences the + sign can also be used in DTD.

Review Questions

1. Discuss the applications of CFG in XML.

SPPU : May-06, Dec.-06, Marks 4, May-11, Marks 6, Oct.-19, In Sem, Marks 3

2. Explain the application of context free grammar to syntax analysis phase of compiler with suitable example.

SPPU : Dec.-13, Marks 6, Oct.-19, In Sem, Marks 3

3. Explain with examples any two applications of context free grammar.

SPPU : Oct.-16, In Sem, Marks 4

3.16 Case Studies

CFG for Palindrome

- The palindrome is a condition in which we get the same string when read it from left to right or from right to left.
- Let G be CFG such that

$G = (V, T, P, S)$ where

$V = \{S\}$

$T = \{a, b\}$

S is a start symbol.

P - The production rules are

$$S \rightarrow aSb \mid bSa$$

$$S \rightarrow a \mid b \mid \epsilon$$

- The simulation of above CFG can be shown for string abba.

S
a S a
a b S b a
a b ϵ b a
a b b a

CFG for Parenthesis Match

- Parenthesis match is a match made between opening and closing parenthesis.
- Let G be CFG such that

$$G = (V, T, P, S) \text{ where}$$

$$V = \{S\}$$

$$T = \{(,)\}$$

S is a start symbol.

P is a set of production rules.

The rules are

$$S \rightarrow SS \mid (S) \mid ()$$

- The simulation above CFG can be shown for the string (()) ()

S
SS
S()
(S)()
(())()

3.17 Fill in the Blanks

- Q.1** The CFG stands for ____.
- Q.2** In CFG normally the terminals are denoted by ____ case and non terminals are denoted by ____ case.
- Q.3** Ambiguous grammar is context free grammar. (true/false)
- Q.4** In derivation tree the derivation is always read from ____.
- Q.5** Ambiguous if grammar is ____.
- Q.6** For every context free language we can construct ____ to accept the language.
- Q.7** The process in which we eliminate the products which yield no terminal is called removal of ____ symbols.
- Q.8** The intersection of context free language and regular language is ____.
- Q.9** ____ is used to prove a language is not context free.
- Q.10** $a^i b^i c^i \mid i \geq 0$ is context free language (True/False)
- Q.11** The pumping lemma is often used to prove that language is ____.
- Q.12** ____ is the pumping length of string of length x ____.
- Q.13** Expand CNF ____.
- Q.14** Expand GNF ____.

3.18 Multiple Choice Questions

- Q.1** Context free grammar is ____.
- | | |
|--|--|
| ☐ a compiler | ☐ b language expression |
| ☐ c language translator | ☐ d none of these |
- Q.2** Which of the following does not belong to context free grammar ?
- | | |
|--|--|
| ☐ a Non terminal symbol | ☐ b Terminal symbol |
| ☐ c Start symbol | ☐ d End symbol |
- Q.3** Regular grammar is ____.
- | | |
|---|---|
| ☐ a context free grammar | ☐ b non context free grammar |
| ☐ c simple english grammar | ☐ d none of these |

Q.4 $S \rightarrow aSa|bSb|a|b$; The language generated by the above grammar over the alphabet $\{a,b\}$ is the set of _____.

- ☐ a all palindromes
- ☐ b all odd length palindromes.
- ☐ c strings that begin and end with the same symbol
- ☐ d all even length palindromes

Q.5 The CFG $S \rightarrow aS \mid bS \mid a \mid b$ is equivalent to regular expression ____.

- ☐ a $(a + b)$
- ☐ b $(a + b)(a + b)^*$
- ☐ c $(a + b)(a + b)$
- ☐ d None of these

Q.6 Context free grammar is closed under _____.

- ☐ a union
- ☐ b Kleen star
- ☐ c concatenation
- ☐ d all of them

Q.7 The language $\{ \}$ is _____.

- ☐ a regular
- ☐ b context free but non regular
- ☐ c context sensitive but not context free
- ☐ d none of these

Q.8 A grammar is ____ device.

- ☐ a generative
- ☐ b cognitive
- ☐ c acceptor
- ☐ d can't say

Q.9 The context free grammar $S \rightarrow SS \mid 0S1 \mid 1S0 \mid \epsilon$ generates _____.

- ☐ a equal number of 0's and 1's
- ☐ b unequal number of 0's and 1's
- ☐ c any number of 0's followed by any number of 1's
- ☐ d none of these

Q.10 The Chomsky normal form is _____.

- ☐ a nonterminal $\rightarrow$ nonterminal nonterminal
- ☐ b nonterminal $\rightarrow$ terminal nonterminal
- ☐ c nonterminal $\rightarrow$ nonterminal
- ☐ d none of these

Q.11 The Chomsky normal form is _____.

- ☐ a nonterminal $\rightarrow$ nonterminal non-terminal
- ☐ b nonterminal $\rightarrow$ terminal nonterminal
- ☐ c nonterminal $\rightarrow$ nonterminal
- ☐ d none of these

Q.12 Given a grammar in GNF and a derivable string in the grammar with the length n , any _____ will halt at depth n .

- ☐ a top-down parser
- ☐ b bottom-up parser
- ☐ c multitape turing machine
- ☐ d none of the mentioned

Q.13 In order to simplify a context free grammar, we can skip the following operation :

- ☐ a Removal of null production
- ☐ b Removal of useless symbols
- ☐ c Removal of unit productions
- ☐ d None of the mentioned

Q.14 Which among the following is a unit production ?

- ☐ a $A \rightarrow BB$
- ☐ b $A \rightarrow B$
- ☐ c $A \rightarrow BC$
- ☐ d All of the above

Answer Keys for Fill in the Blanks :

Q.1	Context Free Grammar	Q.2	lower, upper
Q.3	True	Q.4	top to bottom
Q.5	having more than one derivation tree for the given input string	Q.6	push down automata
Q.7	useless	Q.8	context free language
Q.9	Regular closure	Q.10	False
Q.11	non context free language	Q.12	$x+1$
Q.13	Context Free Language	Q.14	Greibach Normal Form

Answer Keys for Multiple Choice Questions :

Q.1	b	Q.2	d	Q.3	a	Q.4	d
Q.5	b	Q.6	d	Q.7	b	Q.8	a
Q.9	a	Q.10	a	Q.11	b	Q.12	a
Q.13	d	Q.14	b				

Multiple Choice Questions Explanation

Explanation : The rules $S \rightarrow as|bS$ generates $(a+b)^*$. In the given list of production rules $S \rightarrow alb$ are the two rules. Hence the given CFG represents the regular expression $(a+b)(a+b)^*$.

□□□